

INSTITUTO FEDERAL DO NORTE DE MINAS GERAIS
CAMPUS MONTES CLAROS
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

**DETECÇÃO DE IDEACÃO SUICIDA
UTILIZANDO CONJUNTOS DE
CLASSIFICADORES**

PATRICK DUARTE PIMENTA
ORIENTADOR: LAÉRCIO IVES SANTOS

Montes Claros

Julho de 2026

PATRICK DUARTE PIMENTA

**DETECÇÃO DE IDEACÃO SUICIDA
UTILIZANDO CONJUNTOS DE
CLASSIFICADORES**

Projeto de Monografia apresentado ao Curso de Graduação em Ciência da Computação do Instituto Federal do Norte de Minas Gerais – Campus Montes Claros, como requisito parcial para a obtenção do grau de Bacharel em Ciência da Computação.

ORIENTADOR: LAÉRCIO IVES SANTOS

Montes Claros

Julho de 2026

Resumo

O suicídio configura-se como um problema de saúde pública global de grande importância. Anualmente, contabiliza-se cerca de 720 mil mortes em todo o mundo, segundo estimativas recentes das Nações Unidas (2025). A ideação suicida é caracterizada por pensamentos e comportamentos que indicam a intenção suicida, e sua identificação precoce representa um desafio crítico para a prevenção. Com a crescente digitalização, redes sociais e fóruns tornaram-se os principais ambientes onde jovens e adultos compartilham seus desabafos, muitas vezes de forma anônima, gerando um volume massivo de dados em tempo real sobre seus estados emocionais.

Neste contexto, abordagens computacionais de inteligência artificial (IA) e processamento de linguagem natural (PLN) têm se consolidado para a emissão de alerta e intervenções precoces. No entanto, a literatura atual concentra-se predominantemente em contextos no idioma inglês, limitando a aplicabilidade dessas tecnologias em outros idiomas, como por exemplo, a língua portuguesa. Além da barreira idiomática, modelos preditivos baseados em um único classificador frequentemente enfrentam dificuldades em lidar com a ambiguidade e a desestruturação da linguagem informal, o que eleva o risco de falsos negativos. Para superar tais limitações, propõe-se a aplicação de conjuntos de classificadores (ensembles), uma abordagem capaz de equilibrar as falhas de modelos individuais e construir uma fronteira de decisão mais adaptável e segura frente a dados ruidosos.

Assim, este trabalho tem como objetivo realizar uma análise experimental comparativa de modelos de aprendizado de máquina baseados em conjuntos (ensembles) na detecção de ideação suicida, aplicados a uma base de dados estruturada em português brasileiro. A metodologia fundamenta-se no pré-processamento de bases de dados textuais extraídas de fóruns (como o *Reddit*) utilizando técnicas de vetorização, seguido pela implementação e comparação direta entre arquiteturas paralelas de *bagging* (como o *random forest*) e métodos sequenciais de *boosting* (como *XGBoost*). Modelos tradicionais isolados (como a máquina de vetores de suporte, árvore de decisão e *naïve Bayes*) são empregados estritamente para o estabelecimento de linhas de base comparativas.

Como resultados esperados, busca-se determinar qual configuração de *ensemble*

atinge o melhor equilíbrio preditivo, através de métricas de avaliação e validação dos modelos, buscando atenuar sobreajuste e mitigando a incidência de falsos negativos. Dessa forma, pretende-se evidenciar a arquitetura mais robusta para lidar com as particularidades e ruídos da linguagem em redes sociais, fornecendo informações para o desenvolvimento de ferramentas computacionais mais precisas para o apoio à triagem em saúde mental.

Palavras-chave: Ideação Suicida, Processamento de Linguagem Natural, Aprendizado de Máquina, Conjuntos de Classificadores, Redes Sociais..

Abstract

Suicide is a global public health problem of great importance. Every year, there are around 720,000 deaths worldwide, according to recent estimates from the United Nations (2025). Suicidal ideation is characterized by thoughts and behaviors that indicate suicidal intent, and its early identification represents a critical challenge for prevention. With increasing digitalization, social networks and forums have created the main environments where young people and adults gather their feelings, often anonymously, generating a massive volume of real-time data about their emotional states.

In this context, computational approaches of artificial intelligence (AI) and natural language processing (NLP) are consolidated for issuing warnings and early interventions. However, current literature predominantly focuses on English language contexts, limiting the applicability of these technologies in other languages, such as Portuguese. In addition to the language barrier, predictive models based on a single classification often face difficulties in dealing with the ambiguity and destructuring of informal language, or which increase the risk of false negatives. To overcome such limitations, we propose the application of ensembles, an approach capable of balancing the failures of individual models and building a more adaptable and safe decision boundary in the face of noisy data.

Thus, this work aims to carry out a comparative experimental analysis of machine learning models based on ensembles in the detection of suicidal ideation, applied to a structured database in Brazilian Portuguese. The methodology is based on the pre-processing of textual databases extracted from forums (such as *Reddit*) using vectorization techniques, followed by the implementation and direct comparison between parallel *bagging* architectures (such as *random Forest*) and sequential *boosting* methods (such as *XGBoost*). Traditional isolated models (such as the support vector machine, decision tree, and *naïve Bayes*) are used for establishing comparative baselines.

As expected results, we seek to determine which *ensemble* configuration achieves the best predictive balance, through model evaluation and validation metrics, seeking to mitigate overfitting and mitigate the incidence of false negatives. In this way, we intend to highlight the most robust architecture to deal with the particularities and

noise of language in social networks, providing information for the development of more accurate computational tools to support mental health screening.

Keywords: Suicidal Ideation, Natural Language Processing, Machine Learning, Classifier Sets, Social Networks..

Sumário

Resumo	v
Abstract	vii
Lista de Figuras	xi
Lista de Tabelas	xiii
1 Introdução	1
1.1 Definição do Problema	3
1.2 Motivação	3
1.3 Objetivos	4
1.3.1 Objetivo Geral	4
1.3.2 Objetivos Específicos	4
1.4 Organização do Texto	5
2 Fundamentação Teórica	7
2.1 Processamento de Linguagem Natural (PLN)	7
2.1.1 Pré-processamento de Dados Textuais	8
2.1.2 Representação Textual Tradicional	11
2.1.3 Representação Textual Baseada em Embeddings	13
2.2 Aprendizado de Máquina (ML)	16
2.2.1 Tipos de Aprendizado	17
2.2.2 Métodos de Classificação	18
2.2.3 Aprendizado Baseado em Conjuntos	23
2.2.4 Aprendizado Profundo	30
2.3 Avaliação e Validação dos Modelos	31
2.3.1 <i>Validação Cruzada</i>	32
2.3.2 Matriz de Confusão	32
2.3.3 Acurácia	34

2.3.4	Precisão	34
2.3.5	Sensibilidade	34
2.3.6	F1-Score	35
2.3.7	Curva ROC e AUC	35
3	Trabalhos Relacionados	37
4	Metodologia	45
4.1	Base de Dados	46
4.2	Pré-processamento de Dados Textuais	46
4.3	Representação Textual (Vetorização)	47
4.4	Treinamento e Otimização dos Modelos	47
4.5	Avaliação e Validação dos Modelos	48
4.5.1	Reprodutibilidade e Ambiente de Execução	49
4.6	Cronograma de Atividades	50
	Referências Bibliográficas	51

Lista de Figuras

2.1	Aprendizado Não Supervisionado	17
2.2	Aprendizado Supervisionado	18
2.3	Exemplo conceitual de um classificador Support Vector Machine	21
2.4	Exemplo de Árvore de Decisão aplicada à classificação de e-mails.	22
2.5	Arquitetura conceitual de métodos de <i>Ensemble</i> Paralelos. Fonte: Adaptado de IBM (2026).	23
2.6	Arquitetura conceitual de métodos de <i>Ensemble</i> Sequenciais. Fonte: Adaptado de IBM (2026).	24
2.7	Arquitetura conceitual do método de <i>Ensemble Stacking</i> (Empilhamento). Fonte: Adaptado de IBM (2026).	25
2.8	Funcionamento do método <i>bagging</i> , demonstrando a amostragem com reposição e a agregação de classificadores. Fonte: Adaptado de IBM (2026).	27
2.9	Funcionamento sequencial do <i>Boosting</i> , demonstrando a re-ponderação contínua dos dados baseada nas previsões incorretas dos modelos anteriores. Fonte: Adaptado de IBM (2026).	28
2.10	Exemplo conceitual de uma Rede Neural Artificial (RNA) para classificação binária.	31
2.11	Representação visual da Matriz de Confusão em quadrantes.	33
2.12	Representação visual de uma Curva ROC e a respetiva Área sob a Curva (AUC), ilustrando a capacidade de discriminação de um modelo preditivo.	36
4.1	Representação visual do fluxo de processamento do projeto.	45

Lista de Tabelas

2.1	Evolução do texto através do <i>pipeline</i> de pré-processamento.	9
3.1	Quadro comparativo entre os trabalhos relacionados.	42
4.1	Cronograma de desenvolvimento do TCC e entregas do projeto.	50

Capítulo 1

Introdução

O suicídio configura-se como um problema de saúde pública global de grande importância, contabilizando cerca de 720 mil mortes anualmente em todo o mundo [Nações Unidas, 2025; Chowdhury et al., 2026]. A ideação suicida é frequentemente definida e difundida como a ocorrência de pensamentos, preocupações incomuns ou planejamentos voltados para o fim da própria vida, sendo representado um estágio comportamental crítico no espectro de risco de suicídio [Turecki et al., 2019; Bashir et al., 2024]. Nem todos os pensamentos suicidas ocasionam suicídio, no entanto, o suicídio é definido como um processo que, primariamente, escala da ideação suicida às tentativas, levando eventualmente ao suicídio consumado, e podendo ser influenciado por fatores de risco, tanto distais (isolamento social, desemprego) quanto proximais (transtornos psicológicos, abuso de substâncias) [Bashir et al., 2024]. Identificar precocemente esses sinais poderia oportunizar intervenções com o objetivo de reduzir o risco do indivíduo de tirar a própria vida [Allam et al., 2025].

Historicamente, a detecção da ideação suicida dependia habitualmente de avaliações clínicas e questionários tradicionais [Liu et al., 2022; Bashir et al., 2024]. Entretanto, com o advento da internet, as redes sociais e fóruns *online* tornaram-se os principais ambientes onde indivíduos podem expressar livremente seus sentimentos e emoções [Trentarossi et al., 2025]. No âmbito digital, a maior predominância de usuários é constituída por adolescentes e jovens adultos, que passam uma parte significativa do seu tempo *online*, interagindo em plataformas que se tornaram locais para entretenimento e socialização [Trentarossi et al., 2025]. Além disso, esses ambientes digitais também atuam como espaços para a expressão de sofrimento, desabafos e ideação suicida por parte dos usuários [Trentarossi et al., 2025]. Consequentemente, o espaço virtual se tornou uma oportunidade valiosa no que diz respeito ao monitoramento e à detecção preditiva da ideação suicida antes que se concretize em comportamentos que podem ocasionar a morte [Gil Arias et al., 2026].

Uma estratégia comumente utilizada para realizar o monitoramento da ideação suicida de forma automática é a utilização de técnicas baseadas em aprendizado de máquina (*machine learning* — ML), em conjunto com as técnicas de processamento de linguagem natural (PLN). Assim, a integração dessas tecnologias permite monitorar um grande número de pessoas por meio de grandes volumes de dados textuais não estruturados oriundos das plataformas digitais [Allam et al., 2025]. Ao contrário de métodos clínicos tradicionais de triagem baseados em autorrelatos, que frequentemente podem subestimar o risco devido ao medo de estigmas ou internação por parte do paciente [Brown et al., 2020; Roaten et al., 2021; Lowry et al., 2024; Shin et al., 2025; Shingekar et al., 2025], os modelos preditivos, objetivamente, conseguem identificar padrões comportamentais, mudanças emocionais e marcadores linguísticos sutis de sofrimento a partir da escrita espontânea dos usuários [Allam et al., 2025]. Além disso, essa abordagem consiste em uma alternativa de baixo custo, escalável e não intrusiva [Blanca Casas & Guibert Reyes, 1998; Gil Arias et al., 2026]. A aplicação dessas ferramentas fornece um sistema de vigilância contínua, podendo ser capaz de auxiliar profissionais da saúde com alerta para intervenções oportunas na prevenção do ato suicida [Allam et al., 2025]. Apesar da importância do monitoramento digital para a prevenção do suicídio, a extração de sentimentos desses textos depende de técnicas de PLN — fundamentais para o pré-processamento do bloco textual, de maneira que o computador entenda e estruture a linguagem humana [Hameed et al., 2025].

Entretanto, o uso de abordagens que utilizem apenas um único algoritmo classificador frequentemente enfrenta limitações técnicas [Liu et al., 2022; Bashir et al., 2024]. Modelos de classificadores tradicionais, ao operarem de forma isolada, apresentam dificuldades em capturar interpretações distorcidas, além de ambiguidades e especificidades da linguagem humana, sobretudo no cenário em que indivíduos estão em sofrimento emocional [Bashir et al., 2024; Nadeem, 2025; Gil Arias et al., 2026]. Por consequência, modelos de classificadores únicos correm o risco de gerar altos índices de falso negativo, quando o resultado do modelo se apresenta negativo para ideação suicida, porém a condição real do usuário é positiva, o que se configura como uma falha clínica severa e de alto risco em contextos de saúde mental [Bhuiyan et al., 2025; Allam et al., 2025]. A acrescentar, outra limitação pertinente ao estudo de detecção de ideação suicida se dá pela exclusividade idiomática dos modelos. Revisões sistemáticas recentes expõem o viés linguístico na literatura, comprovando que a maioria das pesquisas e treinamentos dos modelos é realizada utilizando base de dados proveniente do idioma inglês [Malhotra & Jindal, 2024; Xu et al., 2024; Qorich & El Ouazzani, 2024; Gil Arias et al., 2026]. Portanto, é evidenciado a escassez de recursos linguísticos que os modelos apresentam ao se tratar em outros idiomas [Zevallos et al., 2025; Zareei et al.,

2025]. O português brasileiro, por exemplo, dispõe de poucos recursos anotados (como dados rotulados manualmente) quando comparado ao idioma inglês, o que dificulta o treinamento de modelos eficazes [Souza et al., 2020].

O estado da arte indica que a adoção de métodos de aprendizado de máquina baseados em conjuntos (*ensembles*) consiste em uma estratégia promissora [Liu et al., 2022], integrando as previsões de múltiplos modelos de classificação básicos para alcançar uma decisão final mais acurada [KINA et al., 2026]. Esta técnica apresenta melhores resultados em comparação com modelos de classificação individuais, podendo reduzir o sobreajuste, além de melhorar o desempenho geral do modelo [Liu et al., 2022]. É importante também citar que na arquitetura *ensemble*, ao juntar classificadores (como, por exemplo *random forest*, regressão logística e SGDC), isto permite capturar padrões textuais complexos e lidar de maneira eficaz com dados ruidosos, o que torna eficaz ao cenário de detecção de ideação suicida, no contexto de aplicação para uma base de dados oriunda de redes sociais (como *Reddit*) [KINA et al., 2026].

No entanto, embora a literatura aponte as vantagens da agregação de modelos, a maioria das pesquisas utiliza os *ensembles* de forma secundária ou junto a modelos mais complexos, como *deep learning*. Por isso, nota-se a escassez de estudos que conduzem uma análise sistemática, isolando e comparando puramente as principais famílias tradicionais de *ensembles*, avaliando-as sob uma etapa padronizada de pré-processamento de texto.

1.1 Definição do Problema

Diante das limitações apresentadas, este estudo orienta-se pelo seguinte problema de pesquisa: como as diferentes arquiteturas de conjuntos de classificadores (*ensemble*) se comportam na detecção de ideação suicida em textos de redes sociais no idioma português, e qual configuração preditiva apresenta a maior eficácia e estabilidade na minimização de falsos negativos?

1.2 Motivação

A motivação central deste trabalho consiste na necessidade de preencher essas lacunas técnicas e linguísticas, investigando qual arquitetura atinge o melhor equilíbrio entre precisão preditiva na redução de falsos negativos e viabilidade técnica aplicada ao contexto do português brasileiro. Por fim, fornecer informações importantes para o desenvolvimento de ferramentas computacionais precisas e robustas para o apoio à triagem de saúde mental no ambiente digital [KINA et al., 2026].

Além disso, torna-se importante destacar que modelos baseados em aprendizado profundo (*deep learning* — DL) apresentam alto desempenho na classificação de textos, no entanto, sua arquitetura complexa frequentemente exige recursos computacionais extensos e infraestrutura de alto custo para o processamento de dados [Allam et al., 2025]. Em comparação com abordagens de DL, a técnica de ensemble motiva-se por ser uma alternativa viável, entregando um nível de acurácia preditiva comparável em diversos cenários da literatura, com a grande vantagem de operar sob baixa complexidade algorítmica e menor custo computacional [KINA et al., 2026].

1.3 Objetivos

1.3.1 Objetivo Geral

Realizar uma análise experimental comparativa de arquiteturas de aprendizado de máquina baseadas em conjuntos (ensembles) para a detecção de ideação suicida, utilizando a base de dados pública Boamente System, composta por instâncias textuais em português brasileiro extraídas de redes sociais. O estudo visa determinar qual configuração preditiva apresenta o melhor desempenho, sendo este medido quantitativamente através dos critérios de acurácia, precisão, sensibilidade, F1-Score e área sob a curva ROC (AUC).

1.3.2 Objetivos Específicos

Para o alcance do objetivo geral, foram definidos os seguintes passos metodológicos:

- Estruturar a representação computacional de textos extraídos de redes sociais em português brasileiro, empregando técnicas de pré-processamento de PLN e métodos de vetorização, a fim de converter as postagens estruturadas em vetores numéricos.
- Treinar classificadores tradicionais isolados para servirem como linhas de base (baselines) na avaliação e na comparação de desempenho preditivo.
- Modelar e otimizar arquiteturas ensembles com o intuito de adequar o modelo computacional ao cenário da base de dados, empregando técnicas para lidar com o desbalanceamento das classes.
- Comparar o desempenho preditivo dos algoritmos por meio de validação cruzada estratificada e testes estatísticos, analisando métricas de avaliação para identificar a arquitetura capaz de minimizar a incidência de falsos negativos.

1.4 Organização do Texto

Para orientar a leitura, este documento está organizado da seguinte forma: o Capítulo 1 apresenta a contextualização do tema, a definição do problema, a motivação e os objetivos da pesquisa. O Capítulo 2 aborda a fundamentação teórica, expondo os principais conceitos necessários para a compreensão do estudo. O Capítulo 3 discute os trabalhos correlatos, situando a presente proposta em relação à literatura existente. O Capítulo 4 detalha o método de pesquisa empregado e a estrutura experimental adotada.

Capítulo 2

Fundamentação Teórica

2.1 Processamento de Linguagem Natural (PLN)

O processamento de linguagem natural (PLN) (ou tratamento da linguagem pelo computador) é um subcampo da inteligência artificial que combina ciência da computação, matemática e linguística [Steinkamp & Cook, 2021; Hameed et al., 2025].

Seu principal objetivo consiste em converter a linguagem humana em instruções computacionais executáveis, facilitando o gerenciamento e a análise de grandes volumes de dados textuais não estruturados, permitindo a extração de informações, análise de sentimentos, detecção de emoções e monitoramento da saúde mental [Hameed et al., 2025]. Os recentes avanços nos modelos de aprendizado computacional permitiram a adoção ampla de técnicas de PLN em diversos domínios, abrangendo desde o comércio eletrônico e a análise de mercados financeiros até o monitoramento em saúde pública [Allam et al., 2025]. Exemplos notáveis são:

- **Agentes Conversacionais (Chatbots):** são sistemas interativos projetados para simular uma conversa humana. Eles são utilizados frequentemente para prestar suporte automatizado e interativo em tempo real nas plataformas digitais [Elsayed et al., 2024; Allam et al., 2025].
- **Modelagem de Tópicos:** são algoritmos probabilísticos que realizam uma varredura em milhares de documentos, agrupando palavras para descobrir temas ocultos [Hameed et al., 2025].
- **Análise de Sentimentos e Emoções:** refere-se à identificação e extração de sentimentos e emoções em blocos de texto, sendo fundamental para captar o estado emocional de usuários em fóruns e redes sociais [Joinson et al., 2024; Allam et al., 2025; Trentarossi et al., 2025; Gil Arias et al., 2026; Rabani et al., 2021].

- **Monitoramento em Saúde Mental:** refere-se ao uso de técnicas de extração de características textuais que permitem rastrear marcadores linguísticos relacionados a transtornos depressivos e à ideação suicida, fornecendo *insights* para o apoio de intervenções clínicas precoces [Allam et al., 2025].

No entanto, desafios são apresentados por essa técnica. Diferentemente dos seres humanos, que possuem a capacidade de interpretar contextos, inferir deduções lógicas e compreender entonações, os computadores dependem exclusivamente de dados estruturados e algoritmos matemáticos para processar informações. Nesse sentido, os sistemas de PLN frequentemente enfrentam dificuldades para compreender as nuances complexas da comunicação humana, sendo propensos a falhar na interpretação de ambiguidades, ironias, sarcasmos e metáforas [Trentarossi et al., 2025]. A utilização de PLN em textos extraídos da internet lida com um obstáculo adicional: a presença de dados ruidosos, repletos de gírias, erros gramaticais e caracteres irrelevantes. Isso exige etapas rigorosas de limpeza e estruturação antes da fase de análise [Bashir et al., 2024].

2.1.1 Pré-processamento de Dados Textuais

O pré-processamento de dados textuais representa uma etapa fundamental de PLN. Os dados provenientes de redes sociais e fóruns da internet são, por natureza, desestruturados e repletos de ruídos, podendo conter caracteres e elementos irrelevantes que não possuem valor semântico [Bashir et al., 2024].

O objetivo central do pré-processamento consiste na transformação de informações brutas em um formato estruturado. Isso garante maior clareza e consistência, permitindo que a máquina consiga interpretar os dados textuais adequadamente [Trentarossi et al., 2025]. Ao reduzir os ruídos, aprimora-se a qualidade dos dados de entrada, possibilitando que o computador consiga identificar tendências e a extrair características relevantes [KINA et al., 2026].

Para que o texto atinja o formato ideal antes de ser compreendido pela máquina, o conteúdo bruto coletado é submetido a um fluxo sequencial de transformações. A literatura especializada aponta que as principais fases de pré-processamento envolvem a limpeza de dados (como a remoção de pontuações, caracteres especiais e links), a tokenização, a remoção de *stopwords* e a normalização morfológica [Trentarossi et al., 2025; Allam et al., 2025; KINA et al., 2026]. Este processo está detalhado nas etapas a seguir.

Para fins didáticos, a Tabela 2.1 demonstra a evolução da frase fictícia “*Estou sofrendo, não aguento mais! Já faz 2 meses.*” ao passar por toda a etapa de pré-

processamento.

Tabela 2.1. Evolução do texto através do *pipeline* de pré-processamento.

Etapas de Pré-processamento	Resultado do Texto
Texto Original	<i>“Estou sofrendo, não aguento mais! Já faz 2 meses.”</i>
1. Conversão para Minúsculas	<i>“estou sofrendo, não aguento mais! já faz 2 meses.”</i>
2. Remoção de Pontuação	<i>“estou sofrendo não aguento mais já faz 2 meses”</i>
3. Remoção de Números	<i>“estou sofrendo não aguento mais já faz me- ses”</i>
4. Tokenização	<i>[“estou”, “sofrendo”, “não”, “aguento”, “mais”, “já”, “faz”, “meses”]</i>
5. Remoção de <i>Stopwords</i>	<i>[“sofrendo”, “não”, “aguento”, “meses”]</i>
6. Normalização (<i>Stemming</i>)	<i>[“sofr”, “não”, “aguent”, “mes”]</i>

2.1.1.1 Conversão para Minúsculas

Como os modelos computacionais probabilísticos costumam ser *case sensitive* (sensíveis a maiúsculas e minúsculas), a etapa inicial da limpeza de dados consiste em converter todo o texto das postagens para letras minúsculas. Essa padronização evita que o modelo trate a mesma palavra escrita com capitalizações diferentes (como "Infelicidade" e "infelicidade") como variáveis distintas, o que prejudicaria a modelagem [KINA et al., 2026].

2.1.1.2 Remoção de Pontuação

Na segunda fase, diversos sinais de pontuação (como \$, %, #, !, ?, entre outros) são excluídos. Tal remoção justifica-se pelo fato de esses caracteres não fornecerem contribuições semânticas para diversas aplicações de PLN. Além disso, manter a pontuação pode dificultar a capacidade do algoritmo em distinguir adequadamente as palavras de outros símbolos soltos [KINA et al., 2026].

2.1.1.3 Remoção de Números

Em seguida, os caracteres numéricos (dígitos) são removidos dos textos, uma vez que o foco da predição está centrado na extração de sentimentos a partir de dados pura-

mente textuais. Os números não exercem impacto na análise semântica neste contexto. Portanto, devem ser eliminados para refinar a cadeia de caracteres [KINA et al., 2026].

2.1.1.4 Tokenização

Concluída a limpeza dos caracteres indesejados, o texto contínuo precisa ser particionado para que o algoritmo consiga analisar os termos isoladamente. Assim, a tokenização representa o processo computacional de segmentar os dados em unidades individuais e gerenciáveis, tipicamente palavras, denominadas *tokens* [Hameed et al., 2025]. Essa segmentação permite a transição de um bloco de texto não estruturado para um formato estruturado [Trentarossi et al., 2025].

2.1.1.5 Remoção de Stopwords

A etapa seguinte atua sobre a lista recém-criada para remover as chamadas *stopwords*. Tais termos, que incluem preposições, artigos e conjunções, são frequentes na estrutura do idioma, mas carecem de valor analítico para a predição. A sua exclusão elimina elementos irrelevantes do texto e reduz o custo computacional da análise [Palomino & Aider, 2022; KINA et al., 2026].

Cabe ressaltar que, no contexto de saúde mental e ideação suicida, termos de negação (como “não”, “nunca” e “jamais”) devem ser mantidos intencionalmente na lista, pois invertem a polaridade do sentimento da frase. A literatura científica demonstra que a remoção indevida dessas palavras degrada severamente o desempenho dos classificadores [Saif et al., 2014; Jianqiang & Xiaolin, 2017; Palomino & Aider, 2022].

2.1.1.6 Normalização

Por fim, procede-se à normalização dos *tokens* restantes. O objetivo desta fase é consolidar as variações morfológicas de uma mesma palavra em uma representação base única, padronizando os dados para a análise algorítmica [KINA et al., 2026; Hameed et al., 2025]. Para este fim, a literatura destaca a utilização de duas abordagens principais. *stemming* e lematização. Porém, como delimitação metodológica deste estudo, será utilizado a técnica *stemming* [KINA et al., 2026; Trentarossi et al., 2025; Bashir et al., 2024].

Stemming é descrito pelo processo de redução das palavras em sua forma radical por meio de um processo heurístico que recorta sufixos e prefixos [Trentarossi et al., 2025]. Com isto, o texto segmentado em palavras individuais é finalmente reduzido às suas formas fundamentais para, logo a seguir, ser convertido em representações numéricas [Bashir et al., 2024; Hameed et al., 2025].

2.1.2 Representação Textual Tradicional

Após a etapa de pré-processamento, embora que os dados textuais estejam limpos e segmentados, eles ainda se encontram em um formato incompatível para o entendimento da máquina, devido a exigência de entradas numéricas para realizar as operações matemáticas de classificação [Allam et al., 2025; Hameed et al., 2025].

Diante disso, a extração de características (ou vetorização) consiste no procedimento computacional responsável por transformar textos não estruturados em matrizes estruturadas de valores numéricos. Isso permite que os modelos probabilísticos identifiquem padrões de sentimentos [Hameed et al., 2025; KINA et al., 2026]. A literatura especializada aponta diversas metodologias para a representação textual, as quais podem se diferenciar devido sua complexidade algorítmica e profundidade semântica.

No entanto, ainda que métodos tradicionais de representação textual e vetorização como BoW e TF-IDF sejam eficazes para tarefas básicas de PLN (como converter texto em vetores de características e capturar padrões lexicais e frequência de coocorrência de palavras) [Salton & Buckley, 1988; Jurafsky & Martin, 2000; Hameed et al., 2025], eles enfrentam limitações em capturar relações semânticas mais profundas e nuances contextuais [Hameed et al., 2025].

2.1.2.1 Bag of Words (BoW)

Bag of words - BoW (ou saco de palavras) é um modelo que representa uma das abordagens mais fundamentais de PLN, fazendo a representação de dados de texto como uma coleção de palavras, sem considerar a gramática ou ordem das palavras [Harris, 1954; Hameed et al., 2025]. O seu funcionamento consiste na criação de um vocabulário a partir de todas as palavras únicas presentes em um conjunto de dados (o *corpus*). Em seguida, cada documento é representado por um vetor numérico, no qual as dimensões indicam a frequência de ocorrência de cada termo na mensagem [Hameed et al., 2025].

A técnica transforma elementos textuais não estruturados em uma representação matemática, gerando como saída um vetor numérico que registra a frequência de ocorrência das palavras classificadas em relação ao vocabulário do modelo, desconsiderando a sua sequência original no texto [Jordan et al., 2022; Hameed et al., 2025]. Frequentemente, essa abordagem que se baseia-se em frequência pode ser combinada com outras técnicas de ponderação, gerando representações significativas e discriminativas [KINA et al., 2026].

Além disso, BoW representa uma metodologia simples e poderosa quando se trata em lidar de forma ampla em tarefas que envolvem classificação de documentos, análise de sentimentos e recuperação de informações (RI), pois captura efetivamente

a presença de palavras em documentos, sendo o indicativo do seu conteúdo [Manning & Schutze, 1999; Hameed et al., 2025]. No entanto, uma das limitações do modelo é o fato de ele desconsiderar a semântica e o contexto das palavras, ocasionando na perda de informações para a compreensão de nuances linguísticas complexas [Jurafsky & Martin, 2000; Hameed et al., 2025].

Para ilustrar o funcionamento do modelo BoW de maneira prática, considere o seguinte exemplo “*Estou sofrendo, não aguento mais! Já faz 2 meses.*” trabalhado anteriormente, em que, após a etapa de *stemming* e de remoção das *stopwords*, o sistema consolide um vocabulário ordenado contendo quatro raízes de palavras: [sofr, não, aguent, mes,].

Dada uma postagem em rede social com o texto “não aguento sofrer”, o algoritmo isola os radicais úteis e contabiliza a sua ocorrência na frase. Portanto, os radicais “sofr”, “não” e “aguent” aparecem uma vez cada, e a raiz de “mes” não aparece, a postagem é convertida para o vetor numérico [1, 1, 1, 0].

Por outro lado, a frase “Já faz meses que eu não visito e não falo com ele.”, os radicais “sofr”, “aguent” não aparecem em nenhuma vez, porém o radical “não” aparece 2 vezes, e o radical “mes” uma única vez. O vetor resultante será [0, 2, 0, 1].

2.1.2.2 Term Frequency-Inverse Document Fequency (TF-IDF)

O *term frequency-inverse document frequency* (TF-IDF) é uma técnica de análise estatística que combina as medidas de *term frequency* (TF), responsável por medir a frequência de uma palavra dentro de um documento, e *inverse document frequency* (IDF), que quantifica a raridade desse termo em todo o conjunto de documentos analisados [Wu et al., 2008; KINA et al., 2026]. Dessa forma, o valor matemático do TF-IDF aumenta proporcionalmente à frequência com que a palavra aparece em um texto específico, mas é normalizado de acordo com a sua raridade em todo o corpus analisado [Wu et al., 2008; KINA et al., 2026].

O cálculo da vetorização por TF-IDF é estruturado em três equações principais [KINA et al., 2026; Hameed et al., 2025].

1. A frequência do termo (TF) calcula a proporção (divisão) do número de vezes em que o termo (t) aparece no documento ($f_{t,d}$) pelo número total de termos contidos no documento (N_d).

$$TF(t, d) = \frac{f_{t,d}}{N_d} \quad (2.1)$$

2. A frequência inversa do documento (IDF) mede a importância global do termo na base de dados, utilizando uma função logarítmica para determinar a raridade

da palavra [KINA et al., 2026]. o número total de documentos no conjunto de dados *corpus* (N) e o número de documentos que contêm o termo t (df_t):

$$IDF(t) = \log \left(\frac{N}{df_t} \right) \quad (2.2)$$

3. O valor final e discriminativo da característica no vetor numérico é obtido pelo produto direto das duas métricas [Hameed et al., 2025]. Esse cruzamento matemático garante que uma palavra tenha um peso alto apenas se ela for frequente em uma postagem específica, mas rara no *corpus* em geral:

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t) \quad (2.3)$$

2.1.3 Representação Textual Baseada em Embeddings

Apesar da eficácia dos métodos tradicionais de vetorização baseados em frequência (como BoW e TF-IDF), essas abordagens fundamentadas em contagem e estatística frequentemente geram matrizes esparsas de grandes dimensões e falham em capturar de forma eficiente as relações semânticas entre os termos [Hameed et al., 2025]. Para superar essas limitações, surgiram os métodos de *word embeddings* (mapeamento de palavras). Essa técnica de extração de características atua transformando a linguagem de forma a atribuir a cada palavra um vetor denso composto por números reais contínuos [Bashir et al., 2024].

Os *embeddings* representam uma abordagem avançada para superar as limitações enfrentadas pelos métodos clássicos baseados em frequência [Hameed et al., 2025]. Arquiteturas como Word2Vec, GloVe [Pennington et al., 2014; Hameed et al., 2025] constituem como *embeddings* simples, enquanto *transformers* (como BERT) [Vaswani et al., 2017; Devlin et al., 2019; Hameed et al., 2025] constituem como *embeddings* contextuais. Dessa forma, *embeddings* são responsáveis pela representação das palavras em espaços vetoriais densos, incorporando contextos semânticos e sintáticos, o que permite uma melhoria significativa do desempenho para uma ampla variedade de aplicações de PLN [Hameed et al., 2025].

Como delimitação técnica para este trabalho, os modelos baseados em arquiteturas *transformers*, representem o atual estado da arte em diversas aplicações de PLN, optou-se pela adoção de métodos de vetorização tradicionais e *embeddings* estáticos (simples). Essa escolha fundamenta-se, prioritariamente, na viabilidade técnica e na eficiência computacional, visto que a utilização de *transformers* frequentemente exige infraestruturas de *hardware* de alto custo e recursos extensos para o processamento de texto.

2.1.3.1 Word2Vec

Word2Vec um dos algoritmos mais populares e eficientes para a geração de representação vetorial de palavras. Trata-se de um método conceitualmente simples e de alto desempenho, uma vez que utiliza uma rede neural artificial, composta por apenas duas camadas, convertendo os dados textuais em um espaço vetorial [Narynov et al., 2019]. Em outras palavras, o Word2Vec insere uma palavra em um espaço de características, de forma que sua localização é determinada pelo significado, ou seja, palavras que apresentam um significado semelhante são agrupadas, e a distância entre as duas palavras também tem o mesmo significado [Narynov et al., 2019].

A arquitetura de treinamento do Word2Vec é operada fundamentalmente em duas técnicas distintas: o *continuous bag of words* (CBoW) e o *skip-gram* [Xia, 2023]. O CBoW visa prever a palavra que se encontra no meio da frase usando as palavras ao redor, enquanto o *skip-gram* prevê as palavras ao redor de uma determinada palavra [Xia, 2023]. Ambas as técnicas são baseadas em probabilidades de adjacência (ocorrência) das palavras em texto [Bashir et al., 2024].

Estudos demonstraram que algoritmos preditivos, ao empregarem o Word2Vec para a vetorização dos *tokens*, conseguem superar abordagens computacionais convencionais [Bashir et al., 2024]. Dessa forma, ao codificar a similaridade semântica dos termos de forma eficiente em grandes conjuntos de dados [Xia, 2023], o algoritmo permite que os classificadores compreendam o contexto e o significado das sentenças analisadas [Bashir et al., 2024].

Para ilustrar o funcionamento do Word2Vec, imagine um cenário prático de classificação de mensagens. Suponha que um modelo processe um grande volume de textos. Em seguida, ele identifica que as palavras “carro” e “veículo” apareceram frequentemente cercadas pelas mesmas palavras vizinhas pertencentes ao mesmo contexto. A partir disso, o algoritmo agrupará os vetores de ambas as palavras em uma região muito próxima no espaço vetorial. Portanto, se um algoritmo preditivo for treinado para reconhecer um padrão associado à palavra “carro” e, durante a etapa de validação, deparar-se com um texto contendo “veículo”, ele saberá interpretá-lo de maneira adequada, devido a similaridade semântica entre os termos através de sua proximidade matemática, em que a curta distância entre os dois vetores reflete a semelhança dos seus significados [Bashir et al., 2024; Narynov et al., 2019].

2.1.3.2 Global Vectors for Word Representation (GloVe)

Global vectors for word representation (GloVe) é apresentado como outra alternativa de destaque no que diz respeito à obtenção de representações vetoriais densas de palavras [Pennington et al., 2014]. Diferentemente dos métodos tradicionais (como BoW ou

TF-IDF), o GloVe atua capturando e aproveitando as informações estatísticas globais de todo o *corpus* textual analisado [Hameed et al., 2025].

O algoritmo constrói uma matriz global de coocorrência de palavras, contabilizando a frequência com que os pares de termos aparecem juntos em determinados contextos [KINA et al., 2026]. Em seguida, o modelo avalia as proporções (razões) dessas coocorrências para codificar as relações semânticas em um espaço de menor dimensionalidade [Hameed et al., 2025]. Um diferencial matemático nesta etapa é que o modelo pondera os dados estatísticos na matriz de coocorrência utilizando de maneira exclusiva os elementos não nulos, otimizando o treinamento [KINA et al., 2026]. Essa abordagem permite que o modelo preserve subestruturas lineares no espaço vetorial, e viabiliza o raciocínio analógico e captura a similaridade semântica entre as palavras [Hameed et al., 2025].

A literatura aponta que o modelo GloVe atua como uma ponte que preenche as lacunas entre os métodos baseados em contagem de frequência (como o BoW e o TF-IDF) e os modelos preditivos (como o Word2Vec, combinando os pontos fortes de ambas as abordagens [Hameed et al., 2025]. Enquanto o Word2Vec isola e captura o contexto local das palavras por meio de janelas deslizantes, o GloVe integra essa capacidade com as informações estatísticas globais, gerando vetores de palavras mais robustos [Levy et al., 2015; Hameed et al., 2025]. É importante citar também que a capacidade do algoritmo em capturar relações sintáticas e semânticas é aprimorada devido à sua escalabilidade eficiente para grandes conjuntos de dados, sendo ideal para tarefas de classificação que exigem *word embeddings* de alta qualidade [Hameed et al., 2025; Liu et al., 2022].

Deve-se observar que o modelo não só aproxima palavras com significados semelhantes, mas também codifica a relação direcional exata entre elas. Por exemplo, a distância matemática entre os vetores das palavras “rei” e “homem” será iterativamente ajustada para ser aproximadamente igual à distância entre “rainha” e “mulher”. Se um sistema aplicar a operação algébrica simples: $v_{rei} - v_{homem} + v_{mulher}$, o vetor resultante no espaço bidimensional ou multidimensional estará geometricamente muito próximo do vetor exato da palavra v_{rainha} .

No entanto, mesmo que métodos *word embeddings* (como Word2Vec e GloVe) tenham trazido avanços significativos ao projetar palavras em espaços vetoriais densos, a literatura especializada aponta que eles enfrentam limitações estruturais inerentes. Para exemplificar, O GloVe é caracterizado como um modelo de representação estatística [Hameed et al., 2025], isso significa que uma palavra sempre receberá o mesmo vetor numérico, independentemente das variações de contexto em que for empregada na frase [Long et al., 2022; KINA et al., 2026]. Devido a incapacidade de adaptação

contextual, algoritmos baseados no GloVe tendem a perder pistas semânticas vitais, falhando particularmente ao processar textos que contêm fortes emoções ou ao demonstrar baixa sensibilidade a palavras raras que indicam condições de saúde mental [KINA et al., 2026].

Após a aplicação das técnicas de PLN para limpar, estruturar e vetorizar os dados, o texto processado será utilizado como entrada direta para os métodos de aprendizado de máquina. Esses algoritmos atuarão sobre as informações numéricas resultantes na etapa de representação textual (vetorização) para identificar padrões e classificar os dados textuais. O funcionamento e as características desses métodos computacionais serão discutidos detalhadamente na próxima seção.

2.2 Aprendizado de Máquina (ML)

O aprendizado de máquina (*machine learning* - ML) representa um subcampo da inteligência artificial (IA), caracterizado pela capacidade dos algoritmos de aprender padrões por meio de um conjunto de dados, os quais são utilizados para o treinamento do modelo. A partir desse processo, o modelo treinado torna-se capaz de realizar previsões com precisão sobre novas entradas (frequentemente denominadas dados do mundo real).

Um modelo ML realiza a manipulação de dados através de algoritmos especializados, tendo a finalidade de gerar modelos preditivos capazes de realizar tomadas de decisões [Jordan et al., 2022]. A descrever seu processo mecânico, sua entrada de dados é descrita como um conjunto de números que possuem significado prático [Jordan et al., 2022]. A partir desse conjunto de numérico, o algoritmo desenvolvido opera sobre eles, buscando encontrar padrões, para gerar dados de saídas previsíveis [Jordan et al., 2022].

Destaca-se também a grande importância da estatística para algoritmos de ML, sendo utilizada no cenário de ML e correspondendo como alicerce para a análise de dados com o objetivo de fundamentar a tomada de decisões de uma IA [Jordan et al., 2022].

Algoritmos de ML utilizam diferentes lógicas e estratégias estatísticas para analisar os dados, extrair padrões e categorizar as informações [Alsaeedi & Khan, 2019]. Na literatura, dependendo da disponibilidade de dados previamente rotulados, essas abordagens são categorizadas primariamente em duas vertentes: o aprendizado supervisionado e o aprendizado não supervisionado. Embora o campo de estudo englobe outros paradigmas, como o aprendizado por reforço, projetado para sistemas de tomada de decisão sequencial em ambientes dinâmicos, este foge ao escopo desta pes-

quiza, visto que o problema abordado se caracteriza estritamente como uma tarefa de reconhecimento de padrões e classificação em textos estáticos. A seguir, detalham-se as principais características e o funcionamento de cada um desses paradigmas aplicáveis ao estudo.

2.2.1 Tipos de Aprendizado

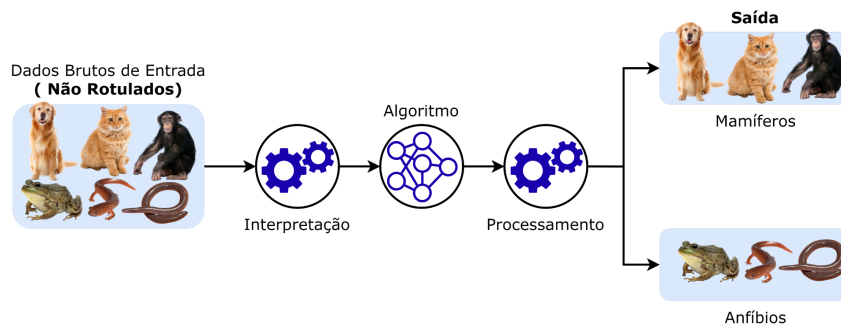


Figura 2.1. Aprendizado Não Supervisionado

O aprendizado não supervisionado representa uma abordagem de ML que lida com dados desprovidos de rótulos prévios. Nessa metodologia, o algoritmo não recebe uma saída esperada durante a fase de treinamento, operando como um modelo de "aprendizado sem professor". Em vez disso, o objetivo central é explorar os dados de forma autônoma para extrair padrões ocultos, identificar estruturas semânticas latentes ou agrupar instâncias que possuam características semelhantes. A Figura 2.1 descreve de maneira simples como um modelo não supervisionado opera diante de um conjunto de dados de entrada brutos e não rotulados.

A literatura destaca que a obtenção de conjuntos de dados rotulados e de alta qualidade exige que especialistas leiam e classifiquem milhares de sentenças, o que torna esse processo exaustivo, dispendioso e trabalhoso, limitando a escalabilidade das pesquisas [Azevedo et al., 2024]. Dessa forma, no cenário de PLN e de análise de sentimentos, o uso do aprendizado não supervisionado torna-se essencial, visto que a análise manual de uma quantidade massiva de informações geradas por humanos e máquinas representa uma tarefa desafiadora [Alsaeedi & Khan, 2019]. A concluir, algoritmos não supervisionados cumprem um papel fundamental para a estruturação e na etapa de descoberta de conhecimento em base de dados informais (que não apresentam rótulos). Isto oferece uma solução robusta para cenários onde a escassez de dados anotados demonstra em ser um obstáculo.

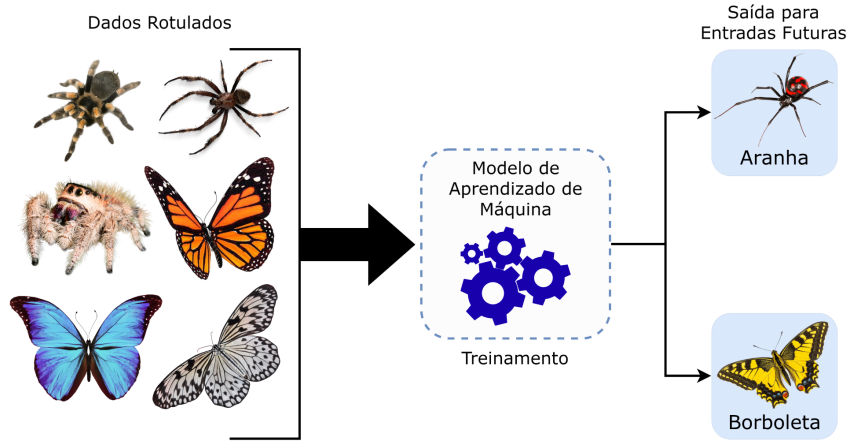


Figura 2.2. Aprendizado Supervisionado

O aprendizado supervisionado representa um paradigma em que o modelo é treinado por meio de um conjunto de dados previamente rotulados. Ou seja, esse processo consiste estritamente na utilização de pares de entradas e saídas, em que as saídas representam variáveis-alvo (entendido também como gabaritos) que o algoritmo deve aprender a prever [Negi et al., 2023]. A Figura 2.2 descreve de maneira simplificada um exemplo de um modelo classificador.

Na etapa de treinamento de um modelo supervisionado, cada exemplo consiste em um vetor de entrada e um valor de saída (rótulo correspondente), isso ajuda o algoritmo a aprender como mapear novos dados para as classes apropriadas [Negi et al., 2023]. Em outras palavras, a finalidade central do algoritmo consiste em aprender como realizar o mapeamento entre as características e os rótulos de saída, para que em seguida, seja capaz de reconhecer padrões e classificar as categorias apropriadas de novos dados genéricos [Matrella et al., 2009; Angiani et al., 2016; Negi et al., 2023].

2.2.2 Métodos de Classificação

Os algoritmos de aprendizado supervisionado são tradicionalmente subdivididos em modelos de regressão e de classificação. Contudo, este trabalho explorará exclusivamente os classificadores, uma delimitação metodológica justificada pela sua ampla aplicação e eficácia no contexto de PLN. Os classificadores são modelos de aprendizado supervisionado que visam realizar previsões para determinar a qual categoria (classe ou rótulo) um novo dado pertence, utilizando como referência um conjunto de dados de treinamento previamente rotulados. Um classificador opera recebendo vetores numéricos de textos, o qual extrai e analisa o conjunto de características (*features*) desses vetores com a finalidade de associar um dado de entrada a uma classe específica (como classificar textos em categorias) [Angiani et al., 2016].

Diversos estudos da literatura empregam de forma ampla esta técnica utilizando classificadores tradicionais (como *naïve Bayes*, *support vector machine* (SVM), árvores de decisão) [Angiani et al., 2016; Jordan et al., 2022] para PNL. Cada uma das técnicas serão detalhadas a seguir.

2.2.2.1 Naïve Bayes

Naïve Bayes é representado como um método de classificação probabilística baseado nos princípios do Teorema de *Bayes*, e sendo utilizado de forma ampla em tarefas de ML [Alsaeedi & Khan, 2019].

A principal característica deste algoritmo, que lhe confere o título de “ingênuo” (*naïve*), é a sua forte premissa de independência condicional entre as variáveis ou características [Alsaeedi & Khan, 2019]. A pegar como exemplo, uma fruta só poderia ser considerada uma maçã se a cor for vermelha, o formato for redondo e ter aproximadamente três polegadas de diâmetro [Alsaeedi & Khan, 2019]. Portanto, independentemente dessas características possuírem correlações reais entre si na natureza ou dependerem da presença de outras características, o classificador as considerará como propriedades estritamente independentes devido à probabilidade de que essa fruta seja uma maçã [Alsaeedi & Khan, 2019].

O princípio fundamental do classificador de *Bayes* é descrito pela fórmula a seguir:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)} \quad (2.4)$$

Onde $P(A|B)$ representa a probabilidade *a posteriori* da classe A ocorrer dado o preditor B ; $P(B|A)$ é a probabilidade do preditor B dada a classe A ; e $P(A)$ e $P(B)$ são as probabilidades *a priori* da classe e do preditor, respectivamente [Alsaeedi & Khan, 2019].

Ao domínio de PLN, a suposição “ingênua” é traduzida em analisar dados textuais sob a forte premissa de que as características extraídas são mutuamente independentes [Angiani et al., 2016]. Portanto, o modelo de *Bayes* assume de forma estrita que a probabilidade de ocorrência de uma palavra específica em um texto não possui absolutamente nenhuma relação ou dependência com a presença de qualquer outra palavra na mesma frase [Gil Arias et al., 2026].

Apesar da premissa de independência ser considerada irreal no mundo real, visto que na linguagem humana, as palavras dependem fortemente uma das outras, o *naïve Bayes* é reconhecido por demonstrar um desempenho que muitas vezes supera estratégias classificatórias mais complexas [Alsaeedi & Khan, 2019]. Devido a sua capacidade

de lidar muito bem com problemas de espaços de dados com alta dimensionalidade, como os grandes vetores gerados pelas representações BoW ou TF-IDF, ele se consagra como um classificador útil para a categorização de textos [Wang & Manning, 2012; Narayanan et al., 2013; Sayfullina et al., 2014].

2.2.2.2 Máquina de Vetores de Suporte (SVM)

Máquina de vetores de suporte (*support vector machine* - SVM) representa outra abordagem de aprendizado supervisionado, sendo um algoritmo utilizado principalmente em tarefas de classificação, e aplicado também ao problema de regressão [Cortes & Vapnik, 1995; Hameed et al., 2025]. O principal objetivo do algoritmo consiste em encontrar o hiperplano ótimo que separe ao máximo os pontos de dados de diferentes classes em um espaço multidimensional [Hameed et al., 2025].

O funcionamento do SVM consiste em maximizar a margem entre os pontos mais próximos de classes diferentes em relação ao hiperplano, sendo estes pontos críticos conhecidos como vetores de suporte [Hameed et al., 2025]. Ao afastar essa fronteira e expandir a margem, o SVM reduz as decisões ambivalentes [Alsaeedi & Khan, 2019].

Portanto, ao lidar com conjuntos de dados complexos, onde as classes não são linearmente separáveis, o SVM emprega uma técnica conhecida como *kernel trick* (truque do kernel) [Hameed et al., 2025]. Essa técnica realiza o mapeamento implícito das características de entrada para espaço de dimensões maiores, isto permite que o algoritmo encontre fronteiras de decisões não lineares de forma eficiente [Hameed et al., 2025].

A Figura 2.3 apresenta um gráfico em duas dimensões (x_1 no eixo horizontal e x_2 no eixo vertical) que ilustra o princípio de funcionamento do SVM. As classes 1 e 2 são representadas, respectivamente, pelos pontos circulares azuis e pelos quadrados vermelhos. O hiperplano ótimo ($w \cdot x + b = 0$) é representado pela reta contínua em preto, que o algoritmo calcula para maximizar a separação entre as classes. As linhas tracejadas representam as margens positiva ($w \cdot x + b = 1$) e negativa ($w \cdot x + b = -1$). Os vetores de suporte são os pontos circulados que se encontram exatamente sobre as linhas tracejadas. A distância entre a linha central e cada margem é determinada por $\frac{1}{\|w\|}$. A largura total da margem de separação é $\frac{2}{\|w\|}$.

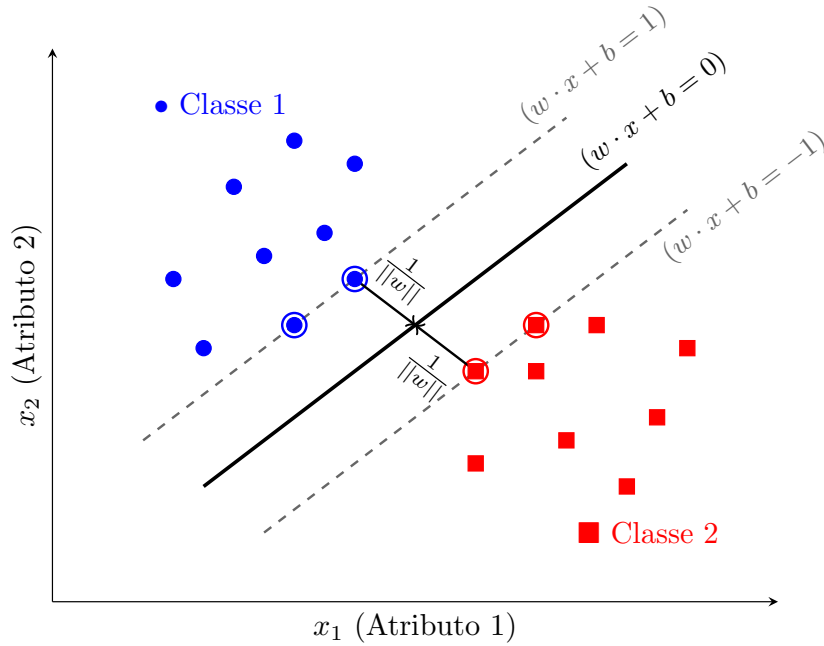


Figura 2.3. Exemplo conceitual de um classificador Support Vector Machine

O SVM se destaca ao contexto de PLN, sendo eficaz quando se trata em trabalhar em espaços de alta dimensão, sendo adequado para a classificação de textos [Hameed et al., 2025]. Além disso, o SVM pode ser combinado com métodos de extração de características (como o TF-IDF) permitindo a classificação de sentimentos em textos [Alsaeedi & Khan, 2019; Joinson et al., 2024]. A literatura também aponta que o SVM apresenta melhor desempenho em comparação com o classificador *naïve Bayes* para diversos problemas de classificação de textos [Alsaeedi & Khan, 2019].

2.2.2.3 Árvore de Decisão (Decision Trees)

Árvore de decisão (*decision tree*) é uma metodologia de aprendizado supervisionado que utiliza uma estrutura em formato de árvore para descrever os dados e resolver problemas tanto de classificação quanto de regressão [Alghazzawi et al., 2025].

O funcionamento mecânico do algoritmo é semelhante à lógica de um fluxograma. Durante a fase de treinamento, o modelo mapeia as decisões computacionais em uma hierarquia estrutura composta por nós (representando testes ou condições sobre as características de entradas) e folhas (representado como a classe final ou resultado da predição) [Jordan et al., 2022]. A Figura 2.4 ilustra o funcionamento de um classificador baseado em árvore de decisão aplicado a um problema clássico de PLN: a filtragem de *spam* em e-mails.

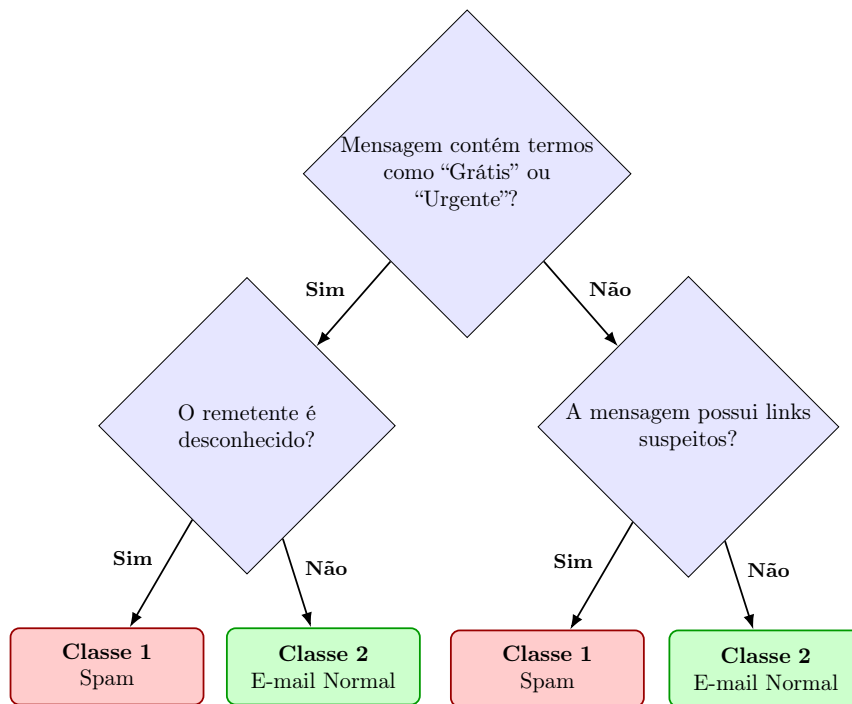


Figura 2.4. Exemplo de Árvore de Decisão aplicada à classificação de e-mails.

O modelo de árvore de decisão estrutura a classificação através de um fluxo de testes sequenciais. O processo começa no nó raiz, que avalia a característica de maior relevância para separar os dados logo de início. A partir dele, os Ramos (caminhos de "sim" ou "não") conduzem aos nós de decisão, que realizam testes intermediários em outros atributos do texto. Esse fluxo contínuo de divisões lógicas segue até alcançar os nós folha, localizados nas extremidades, que representam a predição final do modelo (como classificar um e-mail em "spam" ou "normal").

As principais vantagens das árvores de decisão residem em sua alta interpretabilidade e adaptabilidade [Alghazzawi et al., 2025; Hameed et al., 2025]. Ao contrário de modelos avançados que operam como “caixas-pretas” (*black boxes*), cuja lógica interna é difícil de decifrar, a árvore de decisão permite que os caminhos exatos e as regras lógicas que levaram o modelo a uma determinada classificação sejam rastreados visualmente [Alghazzawi et al., 2025]. Portanto, essa transparência torna-se um componente ético e crítico, pois permite visualizar como a presença ou a ausência de determinadas características lexicais extraídas do texto influenciaram a predição final, facilitando a validação clínica da ferramenta.

2.2.3 Aprendizado Baseado em Conjuntos

O aprendizado baseado em conjuntos (*ensemble learning*) representa uma das estratégias mais avançadas de aprendizado de máquina. Um modelo *ensemble* é descrito pela combinação de múltiplos modelos de classificação individuais, frequentemente denominados como “aprendizes básicos” ou modelos base, com o objetivo de obter previsões finais mais precisas, robustas e generalizáveis. A ideia central desta técnica é definida pela integração de diferentes algoritmos, permitindo explorar os pontos fortes de cada um, atenuando suas limitações individuais e reduzindo problemas clássicos da modelagem, como sobreajuste (*overfitting*) e alta variância [Liu et al., 2022].

A literatura destaca a categorização dos métodos de *ensemble* em dois grupos principais: paralelos e sequenciais.

1. **Métodos Paralelos:** estão relacionados ao treinamento de cada aprendiz básico separadamente (de forma paralela e independente) um do outro. Um exemplo clássico desta abordagem é o algoritmo *random forest*, que cria e treina diversos modelos base aleatoriamente para gerar um preditor mais preciso através da junção de seus resultados individuais [Jordan et al., 2022].

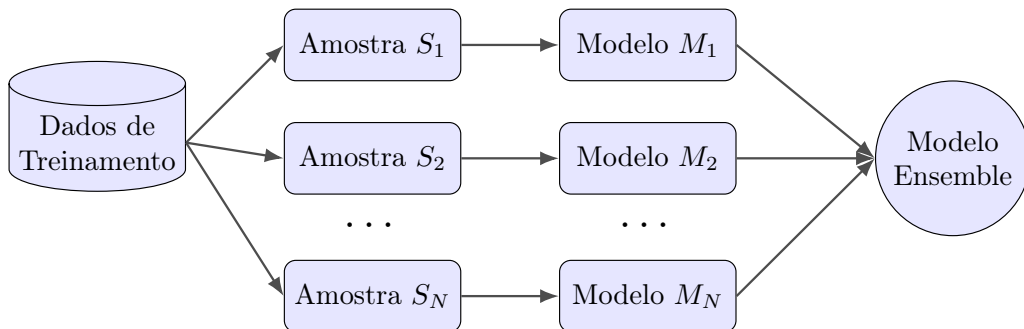


Figura 2.5. Arquitetura conceitual de métodos de *Ensemble* Paralelos. Fonte: Adaptado de IBM (2026).

2. **Métodos Sequenciais:** estão relacionados ao treinamento iterativo, no qual um novo aprendiz básico é construído para minimizar os erros residuais cometidos pelo modelo que foi treinado na etapa anterior. Eles constroem modelos de forma sequencial em estágios, ajustando os pesos das instâncias classificadas incorretamente para que os próximos classificadores se concentrem nos cenários mais complexos [KINA et al., 2026].

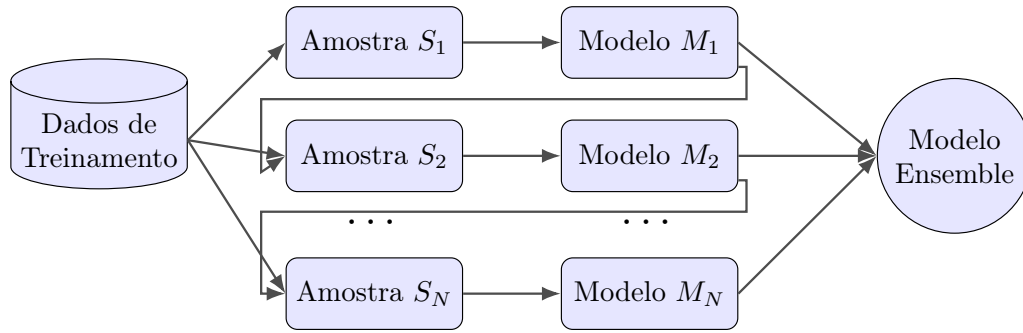


Figura 2.6. Arquitetura conceitual de métodos de *Ensemble* Sequenciais. Fonte: Adaptado de IBM (2026).

Os métodos de *ensemble* são reconhecidos como abordagens altamente eficazes na resolução de problemas complexos de classificação, oferecendo maior estabilidade de desempenho quando aplicados ao cenário de PLN, em comparação com o desempenho de algoritmos isolados [Alsaeedi & Khan, 2019; Bashir et al., 2024].

Portanto, como justificativa para a demonstração, comparação e validação dessa técnica ao presente trabalho, ela se reside pela notável eficiência preditiva que o *ensemble* tem ao agregar as previsões de múltiplos classificadores. Modelos individuais frequentemente encontram dificuldades para capturar as diversas nuances semânticas e estruturais da linguagem humana. Dessa forma, as arquiteturas de *ensemble* constroem uma fronteira de decisão mais equilibrada e confiável, tornando-se a abordagem ideal para tarefas críticas de detecção textual.

2.2.3.1 Votação

A técnica de votação (*voting*) é um método estruturado que permite agregar as saídas de diferentes modelos bases. Em tarefas de PLN, como a análise de sentimentos e classificação de textos, a combinação de classificadores por mecanismos de votação resulta na possibilidade de reunir as características de diferentes modelos, balanceando o modelo preditivo [Bashir et al., 2024; Alsaeedi & Khan, 2019; KINA et al., 2026].

Os métodos de votação são categorizados em dois cenários, dependendo da forma como as saídas dos modelos base são combinadas e processadas: *max voting* e *soft voting* [KINA et al., 2026].

1. **Max Voting (Votação Majoritária):** também conhecida na literatura como *hard voting*, a decisão final do conjunto é determinada pela classe que recebe o maior número de predições diretas entre os classificadores base [Alsaeedi & Khan, 2019].

2. Soft Voting (Votação Suave): em contraposto com *max voting*, *soft voting* baseia-se em integrar as probabilidades preditivas de diferentes modelos, em vez de considerar apenas o rótulo de classe final [KINA et al., 2026]. O algoritmo calcula a média das probabilidades estimadas por cada modelo base para as respectivas classes e seleciona a classe que possui a maior probabilidade média resultante [KINA et al., 2026]. A votação suave apresenta maior robustez em diversas ocasiões porque leva em consideração os níveis de confiança (probabilidades) que cada modelo atribui às suas predições [KINA et al., 2026]. Dessa maneira, modelos que apresentam alta certeza em suas classificações exercem maior influência na decisão final do que modelos com predições incertas, otimizando a precisão do conjunto de classificadores [KINA et al., 2026].

2.2.3.2 Empilhamento

O empilhamento, ou *stacking*, consiste em uma técnica de *ensemble* que combina as predições de múltiplos modelos de classificação para gerar uma decisão final otimizada [Alghazzawi et al., 2025]. Diferente dos métodos de votação, que agregam resultados por medida ou contagem majoritária, o empilhamento utiliza uma arquitetura estruturada em camadas, geralmente composta por um conjunto de modelos base (nível 0) e um meta-classificador (nível 1) [Alghazzawi et al., 2025].

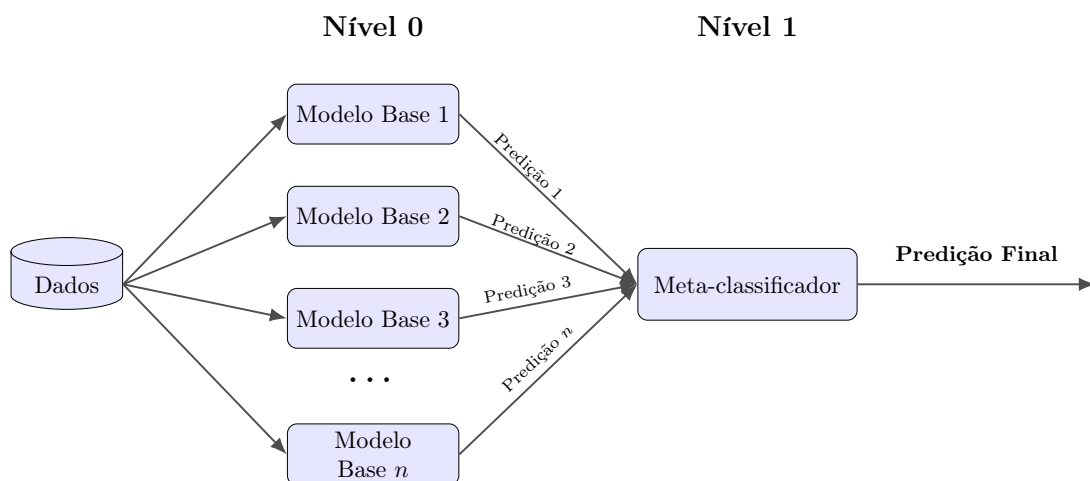


Figura 2.7. Arquitetura conceitual do método de *Ensemble Stacking* (Empilhamento). Fonte: Adaptado de IBM (2026).

No cenário de PLN e na classificação de textos, o processo de empilhamento inicia com o treinamento dos modelos base independentes sobre os dados originais, utilizando as características textuais extraídas (como vetores de frequência ou *embeddings* densos) [Alghazzawi et al., 2025]. As predições geradas por classificadores de nível 0

(como máquina de vetores de suporte e árvores de decisão) não são empregadas como a classificação final [Alghazzawi et al., 2025]. Em vez disso, essas previsões passam a atuar como novos dados de entrada (novas características) para o treinamento do meta-classificador no nível subsequente [Alghazzawi et al., 2025].

A função do meta-classificador se dá em aprender um mapeamento a partir das previsões dos modelos base até os rótulos verdadeiros [Alghazzawi et al., 2025]. Esse processo permite que o meta-classificador (nível 1) identifique quais classificadores base são mais exatos para determinados padrões de linguagem e atue para corrigir os erros individuais de cada um [Alghazzawi et al., 2025].

Além disso, a literatura aponta vantagens de desempenho para a análise de sentimentos e classificação textual quando esta técnica é empregada. A integração de métodos léxicos e classificadores clássicos por meio de empilhamento demonstraram índices de exatidão superiores aos obtidos pela execução de classificadores isolados [Alsaeedi & Khan, 2019]. A principal contribuição do empilhamento, quando utilizado para o processamento de linguagem reside na capacidade de unir perspectivas analíticas diversas sobre os dados textuais, o que compensa as fraquezas específicas que modelos individuais apresentam ao lidar com a desestruturação e a ambiguidade inerentes à comunicação humana [Liu et al., 2022; Alghazzawi et al., 2025].

2.2.3.3 Bagging

Bagging, acrônimo para *bootstrap aggregating*, é definido como outra metodologia *ensemble*, que visa reduzir a variância e mitigar o sobreajuste que algoritmos individuais apresentam [Hameed et al., 2025; Ahmed et al., 2025].

O objetivo desta técnica consiste em gerar múltiplos subconjuntos de dados a partir do conjunto de treinamento original, utilizando amostragem aleatória com substituição. Um modelo base, podendo ser uma árvore de decisão, uma máquina de vetores de suporte ou um modelo *naive Bayes*) é treinado de forma independente em cada um desses subconjuntos. Na etapa preditiva, as saídas de todos os modelos base são agregadas, geralmente por meio de votação majoritária para problemas de classificação.

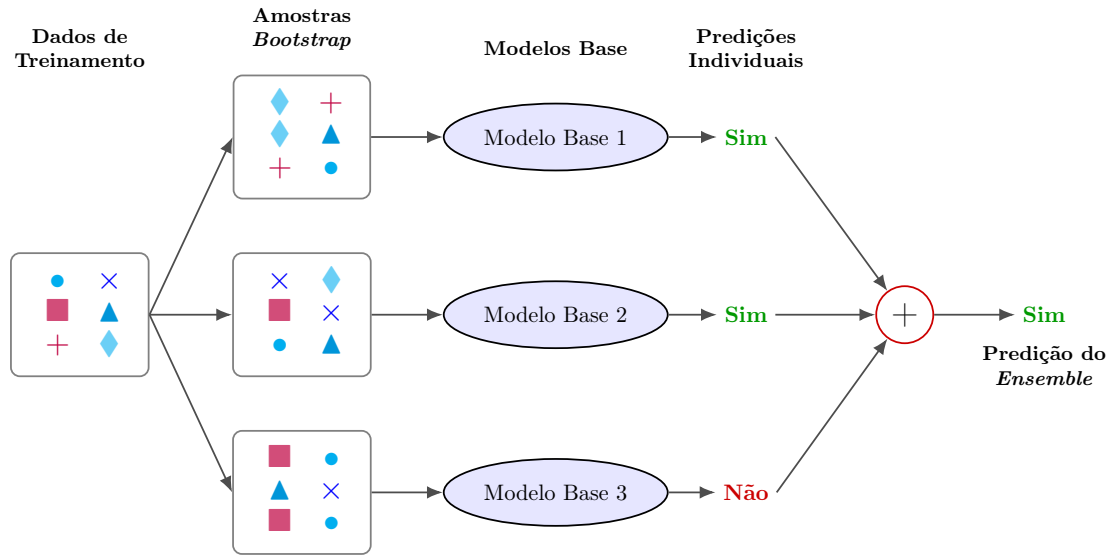


Figura 2.8. Funcionamento do método *bagging*, demonstrando a amostragem com reposição e a agregação de classificadores. Fonte: Adaptado de IBM (2026).

2.2.3.4 Random Forest

Random forest (*floresta aleatória*) é um dos algoritmos mais proeminentes e fundamentados na arquitetura *bagging*. O *random forest* atua construindo uma multiplicidade de árvores de decisão durante a fase de treinamento, e determina o resultado final através da moda das classificações geradas pelas árvores isoladamente [Hameed et al., 2025]. A ideia central desse algoritmo é que a combinação das predições de múltiplas árvores compensa os erros individuais de cada uma delas, o que torna o conjunto robusto perante dados ruidosos [Liaw et al., 2002; Hameed et al., 2025].

No contexto de PLN e tarefas de classificação de textos, a abordagem de *bagging* demonstra utilidade ao processar vetores textuais que geram matrizes de alta dimensionalidade [Hameed et al., 2025; Allam et al., 2025]. A literatura aponta que, ao classificar avaliações de textos de acordo com a sua polaridade, empregando algoritmos de aprendizado supervisionado, foi constatado que o *random forest* obteve desempenho superior na acurácia em comparação a classificadores únicos [Tripathy & Rath, 2017; Alsaeedi & Khan, 2019]. A eficácia do *random forest* na análise de dados baseados em linguagem natural ocorre porque as amostragens aleatórias da técnica de *bagging* impedem que as árvores de decisão memorizem o ruído específico presente em textos [Hameed et al., 2025].

2.2.3.5 Boosting

Boosting (impulsionamento) representa um método iterativo da metodologia *ensemble* que visa criar um modelo preditivo forte a partir da combinação sequencial de múltiplos classificadores fracos [KINA et al., 2026].

Diferentemente de *bagging*, no qual os modelos são treinados de forma independente, na metodologia *boosting*, cada novo modelo base é treinado com o propósito de corrigir os erros cometidos pelos modelos anteriores da sequência [KINA et al., 2026; Hameed et al., 2025].

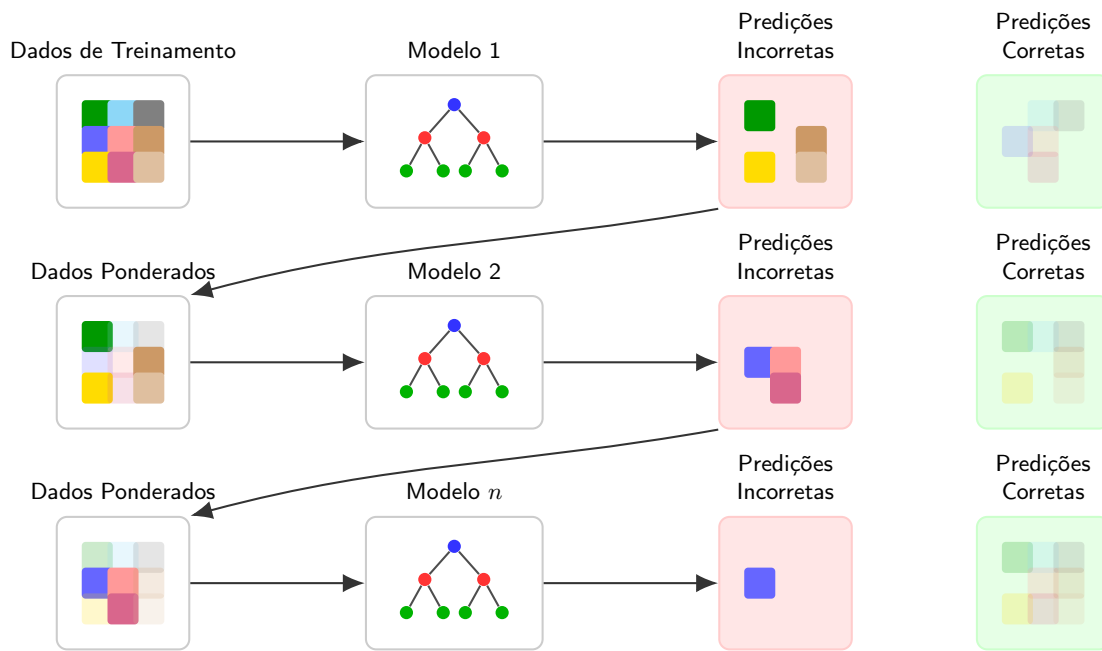


Figura 2.9. Funcionamento sequencial do *Boosting*, demonstrando a reponderação contínua dos dados baseada nas predições incorretas dos modelos anteriores. Fonte: Adaptado de IBM (2026).

Para cada iteração, o modelo *boosting* reajusta o peso dos exemplos de treinamento com base no desempenho obtido pelo modelo atual. Os exemplos que são classificados incorretamente recebem um peso maior para a rodada seguinte [Azevedo et al., 2024]. Portanto, essa característica estrutural faz com que os modelos subsequentes concentrem seu processamento nas instâncias mais difíceis, otimizando de forma gradual a capacidade de generalização do conjunto [Azevedo et al., 2024].

O processo sequencial garante que cada novo modelo fraco treinado atue com o objetivo direto de corrigir os erros cometidos pelos modelos anteriores da cadeia [Azevedo et al., 2024; Hameed et al., 2025]. Portanto, esse ciclo contínuo de avaliação e ajuste persiste até que um critério de parada pré-estabelecido (como um limite de

iterações ou uma taxa de erro aceitável) seja alcançado [Azevedo et al., 2024]. Ao final da execução, a técnica gera um classificador consolidado que reduz o viés e a variância das predições [Azevedo et al., 2024].

2.2.3.6 AdaBoost

AdaBoost (*adaptive boosting*) representa uma das variações da técnica *ensemble* de *boosting*. Atuando como meta-estimador, o algoritmo inicia seu processamento ajustando um classificador base ao conjunto de dados [KINA et al., 2026]. Na literatura de processamento de textos, o classificador base padrão frequentemente utilizado na arquitetura do AdaBoost é a árvore de decisão [Liu et al., 2022].

Esta técnica baseia-se no ajuste de pesos dos dados de treinamento. Após a execução primeiro modelo fraco, o algoritmo treina cópias adicionais do modelo, fazendo a modificação dos pesos das instâncias com base no erro de classificação da iteração anterior: exemplos que foram classificados de forma incorreta recebem pesos maiores [KINA et al., 2026]. Essa abordagem direciona os classificadores subsequentes a concentrarem sua capacidade de predição nos cenários mais complexos da base de dados [KINA et al., 2026]. A decisão final do modelo consolida o resultado de todos os modelos bases treinados ao longo do processo [KINA et al., 2026].

No contexto PLN e na análise de sentimentos, a literatura reporta desempenhos variados para o AdaBoost, a depender das técnicas de extração de características (vetorização) utilizadas [Angiani et al., 2016]. Em tarefas de classificação de textos que utilizam recursos léxicos, o modelo atinge resultados competitivos na categorização das mensagens [Alsaeedi & Khan, 2019]. Apesar de sua utilidade, existem cenários experimentais nos quais o AdaBoost demonstra desempenho preditivo inferior quando comparado a outros métodos de conjunto (como *stacking*) [Liu et al., 2022].

2.2.3.7 Gradient Boosting

Gradient boosting é uma técnica *ensemble*, cuja premissa se descreve pela computação de uma sequência de árvores simples, onde cada árvore sucessiva é construída com o objetivo direto de prever e corrigir erros (resíduos) de classificação gerados pela árvore precedente [Sayfullina et al., 2014; Alghazzawi et al., 2025].

A otimização do modelo é descrita pela técnica *gradient descent* (descida de gradiente), que minimiza uma função de perda de maneira iterativa, fazendo o ajuste do sistema *ensemble* de forma contínua para encontrar soluções classificatórias mais adequadas [Alghazzawi et al., 2025]. Aplicado ao contexto de PLN e na análise de sentimentos, o algoritmo recebe características extraídas dos textos (como representações vetorizadas por frequência) para categorizar as mensagens [Alghazzawi et al., 2025].

O processo de correção sequencial atrelado ao algoritmo permite que o modelo assimile padrões textuais complexos e eleve o poder preditivo global do sistema *Ensemble* [Alghazzawi et al., 2025].

Além disso, a literatura destaca que o *gradient boosting* se consolida como um dos métodos mais robustos para a mineração de dados preditiva [Sayfullina et al., 2014]. Na classificação de textos, a aplicação deste algoritmo gera resultados preditivos consistentes, superando a capacidade de classificadores básicos quando operados de forma isolada [Narynov et al., 2019; Liu et al., 2022]. O modelo também demonstra utilidade prática ao ser incorporado como estimador base em arquiteturas de conjunto heterogêneas, como o empilhamento, isto contribui para a redução da variância e para o ganho de precisão nas predições finais [Alghazzawi et al., 2025; Liu et al., 2022].

2.2.3.8 *XGBoost*

Extreme gradient boosting (XGBoost) consiste em uma implementação avançada do método de *gradient boosting*, projetada para otimizar a eficiência computacional, a escalabilidade e o desempenho preditivo de modelos de ML [Chen & Guestrin, 2016; Hameed et al., 2025].

Como diferencial em sua arquitetura, o XGBoost introduz um termo de regularização matemática projetado de forma específica para mitigar o problema de sobreajuste durante a fase de treinamento [Hameed et al., 2025]. Adicionalmente, a ferramenta possui mecanismos internos estruturados para processar dados esparsos e valores ausentes de maneira eficiente [Hameed et al., 2025]. Essa característica o torna adequado para tarefas de PLN e classificação de textos, uma vez que as representações vetoriais baseadas em frequência (como as matrizes geradas pelo método TF-IDF) costumam gerar espaços de características com alta dimensionalidade e com elevado grau de esparsidade [Hameed et al., 2025].

2.2.4 Aprendizado Profundo

O aprendizado profundo (*deep learning* - DL) é uma subárea avançada de ML, cujos modelos computacionais são inspirados na estrutura e no funcionamento de redes neurais biológicas constituída de células nervosas chamadas neurônios [Hameed et al., 2025]. Diferentemente de modelos de aprendizado supervisionado, que muitas vezes, dependem de uma extração manual de características baseadas em frequência, as redes neurais artificial (RNA) são capazes de realizar a extração hierárquica de atributos de forma autônoma, construindo representações complexas de alto nível a partir de dados brutos e não estruturados [Hameed et al., 2025].

Uma rede neural artificial (RNA) é constituída de camadas de nós interconectados (chamada de neurônios), que processam e transmitem as informações. Inicialmente, essas redes são organizadas em uma camada de entrada, uma ou mais camadas internas (chamada de ocultas) e uma camada de saída [LeCun et al., 2015; Hameed et al., 2025].

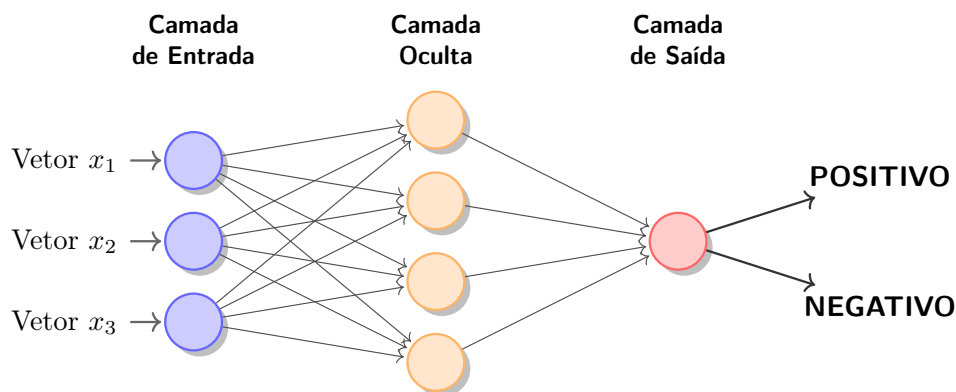


Figura 2.10. Exemplo conceitual de uma Rede Neural Artificial (RNA) para classificação binária.

A figura 2.10 ilustra a arquitetura básica de uma rede neural artificial (RNA). A camada de entrada recebe como por exemplo, os vetores numéricos densos que representam matematicamente as palavras de um texto. A seguir, as informações são repassadas para as camadas ocultas, onde os neurônios aplicam funções de ativação não lineares para realizar a extração hierárquica de características, identificando de forma autônoma padrões semânticos e contextuais complexos [Hameed et al., 2025]. Por fim, as características filtradas chegam à camada de saída, que utiliza funções estatísticas de ativação (como a função sigmoide ou *softmax*) para emitir um escore de probabilidade, classificando o texto nas categorias finais da predição (como, por exemplo, “positivo” ou “negativo” em uma tarefa de análise de sentimentos) [Bashir et al., 2024; Bhuiyan et al., 2025].

Os algoritmos de *deep learning*, quando aplicados em PLN utilizam matrizes densas e vetores contínuos de palavras como sua entrada de dados principais. Ao processar essas representações matemáticas, os modelos de *deep learning* têm demonstrado um desempenho superior na classificação de textos e na extração de padrões semânticos, superando abordagens clássicas e estatísticas [Bashir et al., 2024].

2.3 Avaliação e Validação dos Modelos

A etapa de avaliação e validação é um procedimento importante no desenvolvimento de sistemas de ML. O seu propósito consiste em mensurar a capacidade de generalização de

um modelo preditivo, permitindo verificar o comportamento do algoritmo ao processar dados não observados durante a fase de treinamento [Hameed et al., 2025; KINA et al., 2026].

O processo de validação auxilia na identificação e na mitigação de anomalias de aprendizado, como o sobreajuste (quando o modelo se ajusta excessivamente aos dados de treinamento), assegurando que o classificador aprenda os padrões subjacentes da linguagem em vez de memorizar o ruído presente nas instâncias de treino [Hameed et al., 2025].

Para que a análise seja conduzida de maneira empírica e objetiva, a literatura emprega o uso de técnicas estruturadas de partição de dados e a aplicação de métricas quantitativas de desempenho preditivo, as quais são calculadas a partir das distribuições de acertos e erros geradas pelo modelo [Hameed et al., 2025].

2.3.1 Validação Cruzada

A validação cruzada (*cross-validation*) é uma técnica de reamostragem de dados utilizada para avaliar a capacidade de generalização de um modelo preditivo. O método estima o comportamento do modelo ML ao processar dados inéditos [Alghazzawi et al., 2025]. A validação cruzada em k vias (*k-fold cross-validation*) utiliza todo o conjunto de dados. A técnica divide o conjunto de aprendizagem em k partes (ou *folds*) com proporções iguais [Alghazzawi et al., 2025].

O método funciona por meio de um processo iterativo no qual o modelo é treinado e testado k vezes [Alghazzawi et al., 2025; Yang et al., 2025]. Em cada iteração, uma partição distinta é reservada para atuar de forma exclusiva como o conjunto de testes (validação), enquanto as $k-1$ partições restantes são agrupadas para compor o conjunto de treinamento. Esse ciclo de rotação se repete até que todas as k partições tenham sido utilizadas exatamente uma vez para a validação do modelo [Alghazzawi et al., 2025; Yang et al., 2025].

O desempenho global do classificador é determinado pelo cálculo da média das métricas de avaliação registradas ao longo de todas as iterações executadas [Alghazzawi et al., 2025]. Além disso, é comum utilizar valores usuais para a divisão de conjunto, como (5-*fold*) e 10 (10-*fold*) para a literatura de PLN [Angiani et al., 2016; Alghazzawi et al., 2025; Yang et al., 2025].

2.3.2 Matriz de Confusão

A matriz de confusão consiste em um modelo tabular (duas dimensões) utilizado em estatística computacional e ML para avaliar o desempenho de algoritmos de classificação

[Jordan et al., 2022]. A sua estrutura traduz o comportamento do modelo preditivo, e fornece uma visão detalhada sobre as previsões corretas e os erros em relação aos rótulos reais dos dados avaliados [Angiani et al., 2016; Jordan et al., 2022; Bhuiyan et al., 2025].

A matriz é composta por quatro elementos matemáticos fundamentais, que formam a base para o cálculo de todas as métricas de avaliação subsequentes [Bhuiyan et al., 2025; Ahmed et al., 2025].

		Classificação Real	
		Positivo	Negativo
Classificação Prevista	Positivo	Verdadeiro Positivo (TP)	Falso Positivo (FP)
	Negativo	Falso Negativo (FN)	Verdadeiro Negativo (TN)

Figura 2.11. Representação visual da Matriz de Confusão em quadrantes.

- **Verdadeiros Positivos (TP - *True Positives*):** ocorre quando o modelo preditivo classifica uma instância positiva de forma correta [Ahmed et al., 2025].
- **Verdadeiros Negativos (TN - *True Negatives*):** ocorre quando o modelo classifica uma instância negativa de forma correta [Ahmed et al., 2025].
- **Falsos Positivos (FP - *False Positives*):** representa um erro de classificação no qual o algoritmo prevê a classe positiva para um dado que, na realidade, pertence à classe negativa [Ahmed et al., 2025].
- **Falsos Negativos (FN - *False Negatives*):** constitui o erro no qual o algoritmo prevê a classe negativa para um dado que, de fato, pertence à classe positiva [Ahmed et al., 2025].

2.3.3 Acurácia

A acurácia constitui uma métrica de avaliação que mensura a proporção de predições corretas realizadas por um modelo de classificação em relação ao número total de instâncias analisadas [Bhuiyan et al., 2025]. O seu cálculo é determinado pela soma das classificações corretas, representadas pelos verdadeiros positivos (TP) e verdadeiros negativos (TN), dividida pelo total de amostras do conjunto de dados, o que engloba os acertos e os erros, ou seja, também os falsos positivos (FP) e os falsos negativos (FN) [Jordan et al., 2022; Bhuiyan et al., 2025].

A acurácia é expressa pela seguinte equação [Bhuiyan et al., 2025]:

$$\text{Acurácia} = \frac{TP + TN}{TP + FP + TN + FN} \quad (2.5)$$

2.3.4 Precisão

A precisão é uma métrica de avaliação utilizada em modelos ML de classificação para medir a qualidade das previsões positivas realizadas pelo modelo [Jordan et al., 2022]. O cálculo desta métrica é obtido pela razão entre os verdadeiros positivos (TP) e o número total de instâncias que o algoritmo previu como pertencentes à classe positiva, o que engloba tanto os acertos preditivos quanto os erros de classificação, ou seja, a soma de verdadeiros positivos e falsos positivos (TP + FP) [Jordan et al., 2022; Bhuiyan et al., 2025; Ahmed et al., 2025].

A precisão é expressa pela seguinte equação:

$$\text{Precisão} = \frac{TP}{TP + FP} \quad (2.6)$$

2.3.5 Sensibilidade

A sensibilidade, também denominado *recall*, avalia a eficiência de um modelo ML em identificar de forma correta as instâncias que de fato pertencem à classe positiva [Jordan et al., 2022]. O cálculo desta métrica é determinado pela razão entre as predições corretas da classe positiva, representadas pelos verdadeiros positivos (TP), e o número total de instâncias que são reais positivas no conjunto de dados, o que corresponde à soma dos verdadeiros positivos e dos falsos negativos (TP + FN) [Jordan et al., 2022; Beríña, 2025; Bhuiyan et al., 2025].

A sensibilidade é expressa pela seguinte equação:

$$\text{Sensibilidade} = \frac{TP}{TP + FN} \quad (2.7)$$

2.3.6 F1-Score

O F1-Score consiste na média harmônica entre as métricas de precisão e sensibilidade [Jordan et al., 2022; Allam et al., 2025]. O cálculo desta medida é obtido pela multiplicação entre a precisão e o sensibilidade, cujo resultado é multiplicado por dois e, em seguida, dividido pela soma de ambas as métricas [Jordan et al., 2022]. Essa métrica garante que tanto a exatidão ao prever a classe positiva quanto a capacidade de não omitir instâncias relevantes sejam consideradas de forma simultânea na avaliação [Bhuiyan et al., 2025].

F1-Score é expresso pela seguinte equação:

$$\text{F1-Score} = 2 \times \frac{\text{Precisão} \times \text{Sensibilidade}}{\text{Precisão} + \text{Sensibilidade}} \quad (2.8)$$

2.3.7 Curva ROC e AUC

A curva de característica de operação do receptor (ROC - *receiver operating characteristic*) consiste em uma ferramenta gráfica utilizada para avaliar a capacidade discriminatória de modelos de classificação binária. A estrutura do gráfico plota a taxa de verdadeiros positivos (TPR - *true positive rate ou sensibilidade*) no eixo vertical em oposição à taxa de falsos positivos (FPR - *false positive rate*) no eixo horizontal, registrando o comportamento preditivo do classificador ao longo de múltiplos limiares de decisão [Narynov et al., 2019; Mokheleli et al., 2025].

A área sob a curva (AUC - *area under the curve*) traduz o comportamento gráfico da curva ROC em uma única métrica escalar, cujo valor varia no intervalo entre 0,0 e 1,0 [Narynov et al., 2019; Singh et al., 2023]. O cálculo da AUC consolida a capacidade geral do algoritmo em separar as categorias analisadas [Narynov et al., 2019; Mokheleli et al., 2025].

Na literatura de ML, é estabelecido que valores de AUC sejam próximos a 1,0 indicam um poder de discriminação superior, sugerindo que o modelo preditivo atua de forma consistente na separação das classes, independentemente do ponto de corte escolhido para a decisão [Narynov et al., 2019; Mokheleli et al., 2025]. Em oposição, um valor de AUC igual a 0,5 denota um desempenho equivalente a um classificador aleatório. A análise conjunta da curva ROC e da métrica AUC proporciona uma validação sobre a estabilidade do algoritmo, auxiliando na comparação da precisão do modelo ao lidar com bases de dados e limiares variados [Jordan et al., 2022].

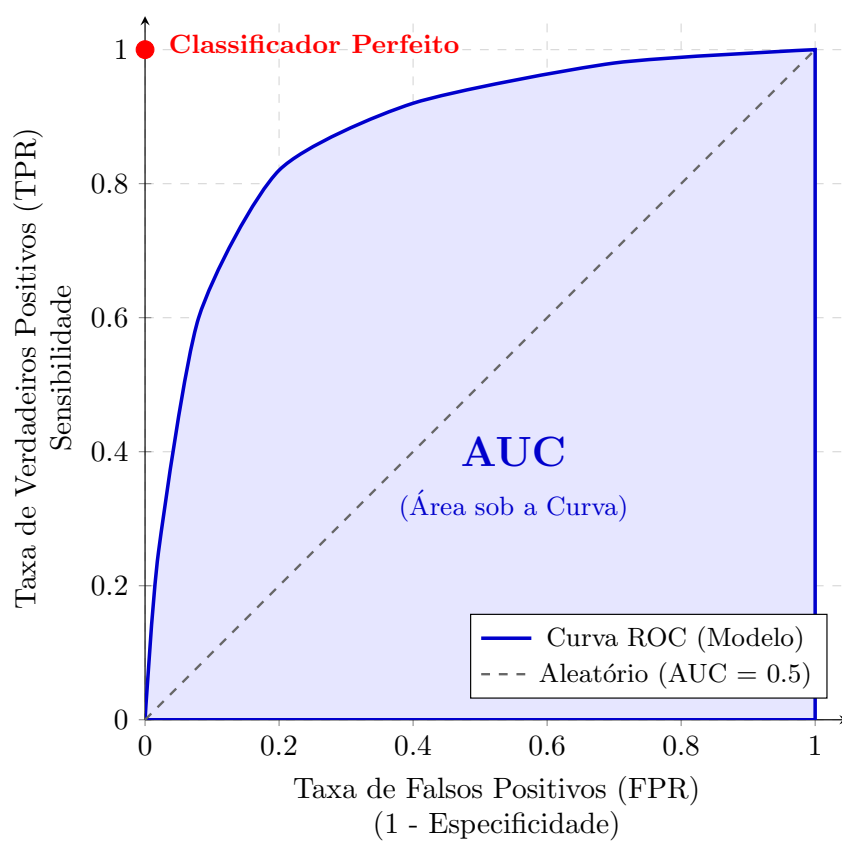


Figura 2.12. Representação visual de uma Curva ROC e a respectiva Área sob a Curva (AUC), ilustrando a capacidade de discriminação de um modelo preditivo.

Capítulo 3

Trabalhos Relacionados

Este capítulo destina-se à apresentação e discussão de pesquisas recentes que utilizam técnicas de processamento de linguagem natural (PLN) e aprendizado de máquina (ML), com ênfase especial em métodos baseados em *ensembles* para a detecção de ideação suicida em textos extraídos de plataformas digitais, como o *Reddit*. Além disso, a partir da revisão da literatura, na qual os principais estudos serão sintetizados na tabela 3.1, no qual os termos em negrito na última coluna identificam o método proposto pelos respectivos autores. O presente texto busca estabelecer uma análise comparativa entre as abordagens (reconhecidas como o estado da arte) e o escopo desta pesquisa. Portanto, essa análise se torna fundamental com a finalidade de identificar e testar as lacunas atuais, cujo tema central deste trabalho está relacionado à demonstração e comparação de métodos ensemble, visando a alternativas mais robustas e precisas para a identificação de ideação suicida.

KINA et al. [2026] propuseram um modelo de *ensemble* baseado em votação suave, denominado SVEM (*soft voting ensemble model*). A arquitetura é composta pela combinação de três classificadores-base: *random forest* (RF), regressão logística (LR) e gradiente descendente estocástico (SGDC). Para a fase experimental, foi utilizada a base de dados disponível no *Kaggle*, composta por 232.074 postagens extraídas no idioma inglês de subfóruns do *Reddit* (*SuicideWatch* e *depression*). Em termos de representação textual, os autores utilizaram técnicas de BoW, TF-IDF e *embeddings* BERT e GloVe. Como destaque, foi utilizada uma abordagem híbrida de engenharia de atributos (WEC), descrita pela união das saídas das técnicas de TF-IDF e BoW. Os autores concluem que a mesclagem dessas técnicas captura a distribuição de frequência dos termos de maneira mais adequada em textos curtos, ruidosos e esparsos, presentes no cenário das redes sociais. Os experimentos avaliaram o desempenho do método SVEM em relação a outras técnicas de aprendizado de máquina, (ML), como classificadores tradicionais, (AdaBoost — ADA, *naïve* Bayes — NB, RF, SGDC, LR),

algoritmos de aprendizado profundo — DL (GRU, CNN, LSTM e RNN), outros *ensembles* (HVEM, SEM, LSTM-GRU-RNN) e IA explicável (LIME). O SVEM obteve melhores resultados em comparação aos modelos de DL, alcançando uma acurácia de 94%. Os autores concluem que o uso de ensembles possibilita a captura de padrões mais complexos devido à diversidade de classificadores fracos e ao baixo custo computacional.

[Alghazzawi et al., 2025] desenvolveram um *framework* baseado em inteligência artificial explicável (XAI), utilizando os métodos SHAP e LIME, e propuseram uma arquitetura *ensemble* para classificar textos de mídias sociais como indicativos ou não de ideação suicida. O modelo *ensemble* de empilhamento estruturado em duas camadas. A primeira camada é composta por quatro algoritmos-base: máquina de vetores de suporte (SVM), regressão logística (LR), *gradient boosting* (GB) e árvore de decisão (DT). Enquanto a segunda camada representa o metamodelo escolhido, o *random forest* (RF). A base de dados utilizada foi extraída do subfórum *Suicide Watch* da plataforma *Reddit* na língua inglesa. Foram analisadas 12.638 instâncias, das quais 6.610 abordam o pensamento suicida e 6.028 tratam de outros tópicos. Na etapa de extração de características dos textos, foram utilizadas técnicas de BoW e TF-IDF. Os resultados obtidos demonstraram que o *ensemble* atingiu uma acurácia média 0,96, e F1-score de 95,5% para a classe suicida. Com isto, o modelo de empilhamento se demonstrou em ter melhores resultados em relação aos algoritmos-base atuando de forma isolada quanto outras abordagens de *ensembles* na literatura. Isto corrobora a eficácia de utilizar conjuntos de classificadores no cenário de ideação suicida, visto que ao agregar múltiplos modelos atenuou as limitações individuais de cada algoritmo, garantindo maior estabilidade e precisão na detecção.

[Bashir et al., 2024] conduziram um estudo com o objetivo de comparar e avaliar o desempenho preditivo de cinco classificadores tradicionais: *naïve Bayes* (NB), *random forest* (RF), árvore de decisão (DT), *gradient boosting* (GB) e XGBoost (XGB); em relação a um algoritmo de aprendizado profundo (LSTM). Além disso, os autores introduziram modelos *ensemble*, gerados pelas combinações desses classificadores, como uma alternativa para aumentar o resultado preditivo por meio do mecanismo de votação. A base de dados utilizada neste experimento foi construída a partir da extração de postagens em inglês dos subfóruns *Suicide Watch* e *depression* do *Reddit*. Foram obtidas 20.000 instâncias, sendo 10.000 postagens referentes à classe de depressão e 10.000 à classe de risco de suicídio. A representação textual foi realizada por meio das técnicas TF-IDF e *Word2Vec*. Os resultados demonstraram que o algoritmo XGBoost, atuando isoladamente, obteve uma acurácia de 86% na base de treinamento, no entanto, na base de testes, seu resultado despencou 61%, evidenciando o problema de sobreajuste.

Em contrapartida, o estudo demonstrou que modelos *ensembles* alimentados por TF-IDF apresentaram acurácias de testes entre 62% e 66%. Como conclusão, o *ensemble* garante um desempenho muito mais robusto e significativo para as classificações das postagens, mitigando de maneira eficiente as fraquezas dos modelos individuais. Um ponto curioso é que neste estudo combinações de Ensemble que incluíam o XGBoost como um dos votantes demonstraram os melhores desempenhos da categoria.

[Azevedo et al., 2024] fizeram um estudo com o objetivo de aprimorar a identificação de ideação suicida em textos não clínicos escritos em português brasileiro, utilizando de métodos de aprendizado computacional semi-supervisionado. Para a fase experimental, os autores utilizaram o conjunto de dados original do sistema *Boamente*, composto por 3.788 postagens do *Twitter* que passaram por rigorosa rotulagem manual por psicólogos clínicos. A fim de contornar a limitação do tamanho da base, o estudo expandiu o escopo realizando a coleta de 11.240 amostras não rotuladas extraídas de subfóruns da rede social *Reddit* (como Ansiedade e Depressão, Transtornos Mentais e Desabafos) no idioma português brasileiro. Em termos de representação textual, o estudo utilizou o modelo de linguagem pré-treinado BERTimbau *large* para a extração contextualizada de características. A arquitetura metodológica baseou-se na aplicação e comparação de cinco técnicas semi-supervisionadas: *self-learning*, *co-training*, *label propagation*, *weighting* e *boosting*. Os resultados revelaram que a aplicação desses métodos impulsionou o desempenho em relação à linha de base do modelo original, com destaque para a técnica de *self-learning* (utilizando um limiar de confiança de 0,8), que alcançou uma acurácia de 99,66%, precisão de 99,54% e uma sensibilidade de 99,27%. Cabe destacar também o notável desempenho do método *ensemble boosting*, que demonstrou alta robustez ao obter 98,30% de acurácia e 98,09% de sensibilidade para os dados no idioma português.

Liu et al. [2022] apresentaram uma arquitetura *ensemble* baseada na fusão de características multidimensionais. Isso significa que, em vez de inserir todos os dados brutos de uma vez em um único modelo, os melhores classificadores-base foram treinados para conjuntos específicos de características. A base de dados utilizada na pesquisa consiste em 40.222 postagens extraídas da rede social chinesa *Weibo*, das quais 2.272 instâncias apresentaram ideação suicida e 37.950 não apresentaram. Para a fusão de características, foram extraídos três conjuntos distintos de informações das mensagens: características estatísticas básicas (BSC), fatores de risco para suicídio (RFS) e agrupamento de *embeddings* de palavras (WEC), representado pela junção do Word2Vec com o algoritmo de agrupamento K-Means. No experimento, os classificadores-base foram testados isoladamente. Na etapa seguinte, os autores aplicaram e compararam diversos métodos *ensemble*, incluindo *random forest* (RF), *gradient boosting* (GB), XG-

Boost (XGB), AdaBoost, *bagging* e empilhamento. Como destaque, o modelo proposto pelos autores utilizou a estratégia de empilhamento aliada à fusão de características, na qual os melhores classificadores-base, especificamente máquina de vetores de suporte (SVM), *naïve Bayes* (NB), regressão logística (LR) e *random forest* (RF), treinados em diferentes subespaços de *features*, alimentam um metaclassificador (regressão logística) para a predição final. Os resultados demonstraram que o modelo proposto obteve o melhor desempenho preditivo, alcançando uma acurácia de 80,61% e F1-score de 79,20%. Isso evidencia que a exploração de múltiplas perspectivas textuais combinada a métodos *ensemble* constrói uma fronteira de decisão mais robusta no contexto clínico, superando abordagens de classificadores únicos.

[Rabani et al., 2021] realizaram um estudo motivado pelo impacto da pandemia de COVID-19 na saúde mental, devido aos altos índices de fatores de risco apresentados nesse período, como o isolamento social, a depressão e a ansiedade. O objetivo do trabalho foi propor uma abordagem baseada em aprendizado de máquina para a detecção e classificação de postagens. Em contraste com o sistema de classificação binária comum nesse contexto (ideação suicida ou não ideação suicida), foram utilizados três níveis de risco e sofrimento: (1) sem risco, (2) ideação suicida de baixo risco e (3) ideação suicida de alto risco. A base de dados utilizada foi construída a partir de duas fontes: X (*Twitter*) e *Reddit*, ambas em língua inglesa. Na primeira fonte, uma API da plataforma foi utilizada para extrair 22.567 instâncias a partir de palavras-chave, frases e *hashtags* relacionadas ao suicídio. Na segunda fonte, foram extraídas 2.676 postagens do *Reddit*, com foco no subfórum de apoio *Suicide Watch*. Para a representação textual, utilizou-se uma engenharia de características que uniu *term frequency-inverse document frequency* (TF-IDF), *bag of words* (BoW), o tamanho da postagem e a modelagem de tópicos baseada em *latent dirichlet allocation* (LDA). Os vetores gerados nessa fase atuaram como entrada para classificadores-base, como máquina de vetores de suporte (SVM), *naïve Bayes* (NB), regressão logística (LR) e árvore de decisão (DT), e para métodos *ensemble*, como *bagging*, AdaBoost (ADA) e *stochastic gradient boosting* (SGB), a fim de classificar os níveis de ideação suicida. Para a validação, o estudo empregou a técnica de validação cruzada de 10 vias (*folds*) para garantir a robustez e diminuir o viés do modelo. Para avaliar o desempenho, foi utilizada a métrica F1-score, como balanceamento de precisão e sensibilidade. Os resultados revelaram que a árvore de decisão e a técnica *bagging* foram os melhores algoritmos para esta tarefa multiclasse, sendo que o primeiro alcançou um F1-score variando entre 96% e 98%, enquanto o segundo obteve valores quase idênticos, variando de 95% a 98%. Com isso, ambos superaram todas as outras arquiteturas testadas no estudo, evidenciando a eficácia do método *bagging* para lidar com dados textuais oriundos de redes sociais.

de Oliveira et al. [2022] conduziram um estudo focado na interpretabilidade e explicabilidade do ecossistema Boamente, utilizando métodos de Inteligência Artificial Explicável (XAI) para desvendar como palavras e sentenças extraídas de textos em português brasileiro influenciam as predições de modelos clássicos de aprendizado de máquina. Os autores treinaram e analisaram três algoritmos sob a perspectiva de interpretabilidade global e local por meio da biblioteca ELI5 (*Explain Like I'm Five*): a Máquina de Vetores de Suporte (SVC), o *Extra Trees* e o *Random Forest*. O pipeline experimental adotou validação cruzada em 5 dobras (*5-fold cross-validation*), particionando os dados em 80% para treino e 20% para teste, sem a aplicação de técnicas de balanceamento de dados. Na avaliação de desempenho geral, os classificadores apresentaram métricas similares, com o *Random Forest* alcançando a maior acurácia (0,906) em comparação ao SVC (0,901) e *Extra Trees* (0,901). Em relação à sensibilidade (*recall*), o *Extra Trees* e o *Random Forest* obtiveram 0,97 na detecção de mensagens sem ideiação suicida, ao passo que o SVC registrou a maior sensibilidade para a classe positiva de ideiação (0,79). No cálculo da área sob a curva ROC (AUC), o SVC apresentou o melhor desempenho médio (0,868), seguido por *Random Forest* (0,864) e *Extra Trees* (0,851). Na análise de explicabilidade global baseada na função *show_weights()*, na qual foram mapeadas 2.965 características, identificou-se que o *Extra Trees* e o *Random Forest* foram igualmente influenciados por termos isolados (unigramas) e compostos (bigramas), enquanto o SVC foi significativamente mais impactado por características de dois termos. Expressões como “suicídio” (*suicidi*), “tristeza” (*tristez*) e variações semânticas de desejo de morrer apresentaram os maiores pesos positivos para indicar a presença de ideiação suicida. Por fim, a análise local conduzida por meio da função *show_prediction()* revelou que o algoritmo *Extra Trees* apresentou maior ajuste à base de dados específica, enquanto o *Random Forest* demonstrou uma capacidade de generalização superior ao processar e classificar amostras de texto com nuances ambíguas ou hiperbólicas.

Diniz et al. [2022] propuseram o ecossistema Boamente, uma solução de fenotipagem digital destinada ao monitoramento inteligente de ideiação suicida em português brasileiro, baseada na coleta passiva de textos digitados pelos usuários por meio de um teclado virtual customizado para smartphones. Para a validação do motor de inferência, os autores coletaram uma base de dados composta por 5.699 postagens não clínicas do Twitter, as quais foram submetidas à rotulação individual de três psicólogos clínicos de diferentes linhas teóricas. Após a eliminação de instâncias com divergências de anotação (1.513 postagens) e a exclusão de 398 duplicatas, o dataset final confiável foi consolidado com 3.788 amostras, das quais 2.691 foram rotuladas como negativas e 1.097 como positivas para ideiação suicida. O processamento textual seguiu um pi-

peline contendo etapas de conversão para minúsculas, limpeza de caracteres especiais, remoção de stopwords, tokenização, stemming e ponderação estatística via TF-IDF. O treinamento envolveu o ajuste de hiperparâmetros por grid search e validação cruzada em 5 dobras, comparando classificadores de aprendizado de máquina clássicos e variantes baseadas em deep learning. Dentre os modelos tradicionais, o Random Forest e o Extra Trees obtiveram as melhores taxas de sensibilidade (recall), registrando 0,895 e 0,877, respectivamente. Contudo, o modelo baseado no transformer BERTimbau Large apresentou o desempenho superior absoluto na tarefa de classificação, alcançando acurácia de 0,955, precisão de 0,961, recall de 0,953 e F1-score de 0,954. Os autores concluíram que o ecossistema Boamente demonstra alta viabilidade técnica e clínica como ferramenta não intrusiva de geração de evidências para apoiar a tomada de decisão de profissionais de saúde mental.

Tabela 3.1. Quadro comparativo entre os trabalhos relacionados.

Trabalho	Idioma da Base de Dados	Representação Textual	Técnica de ML	Comparação Sistemática de Ensembles Tradicionais
KINA et al. [2026]	Inglês.	BoW, TF-IDF, (BoW + TF-IDF), GloVe, BERT.	LR, RF, SGDC, NB, ADA, GRU, CNN, LSTM, RNN, HVEM, SEM, LSTM-GRU-RNN, SVEM e LIME.	Não realizada.
Alghazzawi et al. [2025]	Inglês.	BoW, TF-IDF.	Empilhamento (SVM-LR-GBDT $\rightarrow$ RF) , GBDT-SVM, XGB-CNN, RF-LSTM, SHAP e LIME .	Não realizada.
Bashir et al. [2024]	Inglês.	TF-IDF e Word2Vec.	NB, RF, DT, GB, XGB, NB-RF-DT , NB-RF-XGB , NB-DT-XGB , NB-GB-XGB , RF-DT-GB , RF-DT-XGB , NB-RF-DT-GB-XGB e LSTM.	Não realizada.
Azevedo et al. [2024]	Português Brasileiro.	BERTimbau Large.	Self-learning , Co-training , Label propagation , Weighting e Boosting	Não realizada.

Liu et al. [2022]	Chinês	Fusão de Características : BSC, RFS e WEC (Word2Vec + K-means).	SVM, NB, KNN, LR, DT, RF, XGB, ADA, Bagging e Empilhamento por Fusão ([Modelos Base para BSC, RFS e WEC]. → LR)	Não realizada.
Rabani et al. [2021]	Inglês.	(TF-IDF + BoW + Tamanho da Postagem + LDA).	SVM, NB, LR, DT, Bagging, ADA e SGB.	Parcialmente realizada.
Diniz et al. [2022]	Português Brasileiro.	TF-IDF e representações contextuais internas de Transformers.	RF, Extra Trees, Multilingual BERT, BERTimbau Base e BERTimbau Large (com SMOTE).	Não realizada.
de Oliveira et al. [2022]	Português Brasileiro.	TF-IDF.	SVC, Extra Trees, Random Forest e ELI5 (Interpretabilidade Global e Local) .	Não realizada.

A análise do estado da arte sobre detecção de ideação suicida revela que a adoção de métodos baseados em conjuntos (*ensembles*) tem crescido como uma alternativa robusta e econômica para mitigar a ineficiência e os altos índices de falsos negativos apresentados por classificadores únicos. Estudos como os de KINA et al. [2026] e Alghazzawi et al. [2025] confirmam que a agregação de algoritmos compensa as limitações individuais dos modelos, garantindo maior estabilidade e precisão preditiva.

Contudo, observa-se uma lacuna científica no que diz respeito à profundidade com que esses métodos *ensemble* são comparados de forma sistemática e isolada sob as mesmas condições experimentais. Grande parte da literatura utiliza essas técnicas de forma secundária ou acoplada a arquiteturas complexas. Por exemplo, os estudos de Bashir et al. [2024] e KINA et al. [2026] empregam *ensembles* principalmente para contrastá-los com algoritmos de aprendizado profundo (como LSTM e CNN) ou em arquiteturas de votação e de fusão de características multidimensionais muito específicas. Outros autores focam em estratégias de empilhamento (*stacking*) atreladas à inteligência artificial explicável [Alghazzawi et al., 2025], ou limitam-se a avaliar a interpretabilidade local e global de classificadores clássicos tradicionais atuando de forma isolada [de Oliveira et al., 2022]. Embora o trabalho de Rabani et al. [2021] avalie classificadores tradicionais e *ensembles*, a literatura ainda carece de estudos que isolem e avaliem exaustivamente as principais famílias de conjuntos tradicionais sob as exatas mesmas condições de pré-processamento e vetorização. Adicionalmente, nota-se uma acentuada predominância linguística na literatura, na qual a ampla maioria dos traba-

lhos correlatos foi conduzida com modelos que operam em bases de dados no idioma inglês, com raras exceções como o estudo de Liu et al. [2022] em chinês, e as iniciativas em português brasileiro lideradas por Azevedo et al. [2024], Diniz et al. [2022] e de Oliveira et al. [2022]. Essa forte concentração idiomática evidencia a escassez de recursos linguísticos consolidados (como conjuntos de dados rotulados e mapeamentos de explicabilidade) em outros idiomas, o que reforça a relevância acadêmica e social do presente trabalho ao explorar e otimizar pipelines baseados em conjuntos especificamente voltados para as particularidades semânticas do português brasileiro.

O diferencial deste trabalho consiste justamente em preencher essas limitações, promovendo uma análise sistemática e isolada entre as arquiteturas de votação, empilhamento, *bagging* (como random forest) e as diversas abordagens de *boosting* (como AdaBoost, *gradient boosting* e XGBoosting), cada uma delas operando em uma base de dados do idioma português. A ausência dessa comparação extensiva na literatura impede a identificação de qual arquitetura *ensemble* lida de forma mais robusta com as particularidades léxicas e o ruído presentes em textos oriundos de redes sociais. Assim, objetiva-se investigar qual dessas técnicas atinge o melhor equilíbrio entre precisão preditiva na redução de falsos negativos e baixo custo computacional, sem a necessidade de recorrer a redes neurais — as quais exigem infraestrutura de *hardware* de alto custo e extensa.

O Quadro 3.1 sintetiza essas abordagens, evidenciando as técnicas de representação textual, os algoritmos empregados e o idioma da base de dados utilizado em cada trabalho. A partir dessa síntese visual, a lacuna a ser atacada fica evidente: os estudos listados exploram os *ensembles* de maneira não exaustiva, frequentemente associados a modelos híbridos e de *deep learning*, além de apresentarem uma forte exclusividade idiomática. Ao realizar um estudo comparativo direto focado nas famílias tradicionais de *ensembles*, este trabalho busca consolidar um parâmetro claro sobre o arranjo algorítmico mais eficiente para a detecção de ideação suicida através de textos extraídos de redes sociais, além de fornecer *insights* valiosos para o contexto nacional.

Capítulo 4

Metodologia

Este capítulo destina-se a descrever os procedimentos metodológicos adotados para o desenvolvimento deste trabalho, detalhando fluxo de processamento de aprendizado de máquina estabelecida para a detecção de ideação suicida.

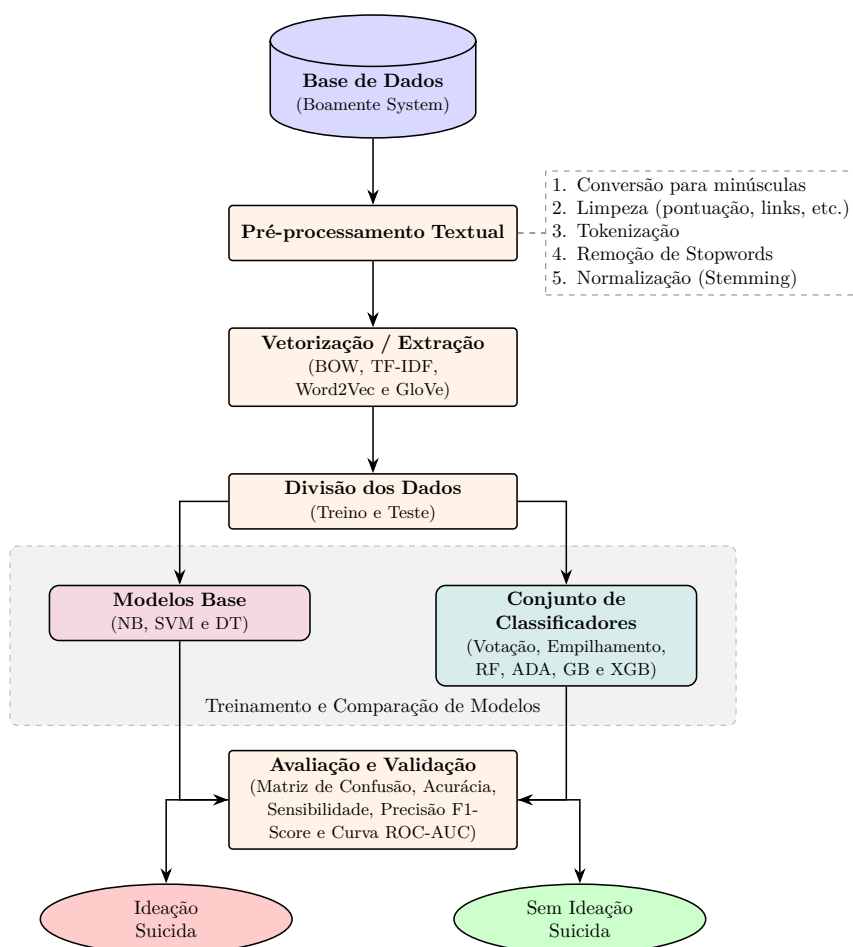


Figura 4.1. Representação visual do fluxo de processamento do projeto.

O fluxo experimental, representado pelo diagrama 4.1, foi desenvolvido com base nos conceitos estabelecidos na fundamentação teórica, englobando as etapas de pré-processamento de dados textuais, representação textual (vetorização), treinamento de classificadores base e de conjuntos de classificadores (*ensembles*), além da avaliação preditiva, representado pelo diagrama.

4.1 Base de Dados

Para a condução deste estudo, será utilizado o conjunto de dados público *Dataset of suicidal ideation texts in Brazilian Portuguese - Boamente System*. Este *dataset* é composto por postagens de redes sociais (como o *Twitter/X*) redigidas em português brasileiro [Azevedo et al., 2024]. Originalmente, o conjunto de dados passou por um processo de rotulação manual especializada, em que três psicólogos clínicos avaliaram as sentenças. Para garantir que os dados sejam confiáveis, e evitar vieses de anotação, as sentenças que apresentaram discordância entre os especialistas foram removidas. O subconjunto resultante confiável de 3.777 instâncias é estruturado em formato tabular, contendo a variável de entrada (*text* em seu formato bruto) e a variável-alvo (*label*), onde 1 indica a presença de ideação suicida (1.090 instâncias), e 0 indica a ausência de ideação suicida (2.687 instâncias).

4.2 Pré-processamento de Dados Textuais

Tendo em vista que textos extraídos da internet e de redes sociais por natureza, apresentam dados ruidosos (contendo gírias, erros e caracteres irrelevantes), a fase de pré-processamento é essencial para estruturar a linguagem de forma compreensível para o modelo computacional. Respeitando o escopo do processamento de linguagem natural (PLN) definido neste trabalho, o texto passará pelas seguintes etapas de limpeza e padronização: (1) conversão do texto para letras minúsculas (*lower casing*); (2) limpeza de caracteres especiais e números; (3) tokenização; (4) remoção de *stopwords*, na qual termos de negação (como “não”, “nunca”, “jamais”, “nada” e “nem”) são preservados, visto que a remoção dessas palavras inverte a polaridade de sentimento dos textos e compromete o desempenho preditivo dos classificadores; (5) e, por fim, a normalização (*stemming*).

4.3 Representação Textual (Vetorização)

Os dados textuais serão convertidos em matrizes estruturadas de valores numéricos para viabilizar as operações matemáticas. Para a representação textual, o método emprega abordagens tradicionais (como BoW e TF-IDF), e baseadas em *embeddings* (como Word2Vec e GloVe). Com o propósito de evitar o sobreajuste decorrente do treinamento de representações vetoriais densas em conjuntos de dados de tamanho reduzido, serão utilizados modelos de *word embeddings* pré-treinados para o idioma português brasileiro, adotando-se os vetores de 300 dimensões disponibilizados pelo repositório do Núcleo Interinstitucional de Linguística Computacional (NILC). Como delimitação técnica estabelecida no referencial teórico, o método descarta a utilização de modelos baseados em arquiteturas *transformers* (como o BERT). Essa escolha fundamenta-se na viabilidade técnica e na eficiência computacional, visto que os *transformers* exigem infraestruturas de hardware de alto custo e recursos extensos, o que foge ao escopo estabelecido para o processamento de texto desta pesquisa.

4.4 Treinamento e Otimização dos Modelos

O treinamento dos modelos será dividido em duas partes: (1) modelos de classificação e (2) modelos *ensembles*. Como justificativa, o treinamento de modelos classificadores está relacionado em estabelecer linhas de base (*baselines*), ou seja, para avaliar a performance preditiva dos modelos *ensemble* por meio de um quadro comparativo. Neste estudo, foram escolhidos três classificadores-base para fase inicial de treinamento, sendo: máquina de vetores de suporte (SVM), árvore de decisão (DT) e *naïve Bayes* (NB).

Como o tema central desta pesquisa reside em realizar uma análise sistemática dos modelos *ensembles*, com a finalidade de investigar e comparar o comportamento preditivo de diferentes arquiteturas no contexto de detecção de ideação suicídio, serão treinados modelos *ensemble*, consistindo: métodos paralelos de *bagging*, como o *random forest* (RF), e métodos sequenciais de *boosting*, como o AdaBoost (ADA), o *gradient boosting* (GB) e o XGBoost (XGB), além das técnicas de votação (*voting*) e empilhamento (*stacking*).

Afim de assegurar o rigor científico na avaliação do desempenho preditivo, este trabalho não se limitará à utilização de hiperparâmetros padrão (*default*) dos algoritmos. Portanto, com a finalidade de otimizar a configuração de cada modelo, o treinamento consistirá na aplicação de técnica de ajuste fino de hiperparâmetros. Especificamente, será empregado o algoritmo *grid search* sob validação cruzada para

mapear sistematicamente o espaço de busca de otimização de cada modelo [KINA et al., 2026]. O *grid search* consiste em um método de busca exaustiva que testa sistematicamente um dicionário contendo diversas combinações de parâmetros (como, por exemplo, a taxa de aprendizado e o número de estimadores no *XGBoost*, ou a profundidade máxima no *random forest*) [KINA et al., 2026]. O objetivo desta etapa é identificar a configuração matemática exata que maximize o desempenho do modelo antes que a classificação final seja computada.

4.5 Avaliação e Validação dos Modelos

Em alinhamento ao objetivo central deste trabalho, a avaliação do poder preditivo dos algoritmos consistirá em uma análise sistemática. O intuito dessa abordagem é avaliar e comparar as diferentes arquiteturas de *ensembles*.

Para garantir a capacidade de generalização e mitigar falhas de sobreajuste frente a dados inéditos, adota-se a técnica de validação cruzada estratificada em k vias (*stratified k-fold cross-validation*), a qual garante a manutenção da proporção original das classes positiva e negativa em cada uma das dobras de treino e validação, o que é conceitualmente indispensável para resguardar o comportamento estatístico de algoritmos em conjuntos de dados clinicamente desbalanceados [Diniz et al., 2022; de Oliveira et al., 2022]. Com o propósito de prevenir a ocorrência de vazamento de dados, que ocorre quando informações estatísticas de validação influenciam indiretamente as variáveis de treinamento, as etapas de pré-processamento, vetorização de termos e eventuais técnicas de reamostragem serão encapsuladas por meio de uma estrutura unificada de *Pipeline* (*scikit-learn*). Esse encadeamento força que as operações de ajuste (*fit*) das características sejam calculadas exclusivamente com base no conjunto de treino de cada iteração, aplicando somente o operador de transformação (*transform*) sobre as amostras de validação isoladas.

O desempenho da classificação será mapeado por meio da matriz de confusão e, a partir desses quadrantes matemáticos, serão extraídas as métricas clássicas de desempenho. Cabe ressaltar que, no cenário de detecção de ideação suicida, a acurácia global desempenha um papel secundário e menos importante de avaliação, visto que o desbalanceamento de classes pode induzir taxas camufladas de falsos negativos em modelos com alta acurácia básica; dessa forma, a validação de excelência dos preditores priorizará os resultados de sensibilidade (*recall*) e F1-score, que medem, de forma direta, a redução segura de omissões e o equilíbrio harmônico das predições positivas. Adicionalmente à análise gráfica gerada pelas curvas ROC e pela correspondente área sob a curva (AUC). Por fim, a validação experimental contará com a aplicação de

testes estatísticos de hipótese, de modo que seja avaliado se as diferenças preditivas observadas entre as combinações por ensemble e os modelos de base tradicionais são cientificamente significativas ou meramente residuais de variações amostrais.

4.5.1 Reprodutibilidade e Ambiente de Execução

Com o propósito de assegurar a total reprodutibilidade científica dos experimentos computacionais e mitigar variações estocásticas nos resultados, todas as etapas de particionamento de dados, geração de amostras sintéticas via SMOTE, inicialização de pesos e treinamento dos algoritmos serão controladas por uma semente de aleatoriedade única (*seed*), definindo-se o parâmetro *random_state* estritamente como 42 em todas as funções correspondentes [Alghazzawi et al., 2025]. Os experimentos serão conduzidos utilizando a linguagem de programação Python (versão 3.12.2) sob o ambiente de desenvolvimento Jupyter Notebook integrado à distribuição Anaconda [KINA et al., 2026; Diniz et al., 2022]. Para o processamento e a limpeza dos textos, empregar-se-á a biblioteca *nltk* (Natural Language Toolkit, versão 3.8.1), responsável pelas etapas de tokenização, remoção seletiva de *stopwords* e redução morfológica por *stemming* [de Oliveira et al., 2022; Diniz et al., 2022]. A modelagem, o ajuste de hiperparâmetros por busca exaustiva em grade (*grid search*) e a avaliação dos modelos serão suportados pelo pacote *scikit-learn* (versão 1.5.0), que fornecerá as implementações nativas do classificador tradicional *Random Forest* e do método de *ensemble AdaBoost*, além das estruturas de pipelines para evitar o vazamento de dados [de Oliveira et al., 2022; Diniz et al., 2022]. O classificador de gradiente sequencial *XGBoost* será implementado de forma otimizada a partir de sua biblioteca nativa independente *xgboost* (versão 2.0.3), garantindo o aproveitamento de recursos de computação paralela de alta performance. Adicionalmente, as bibliotecas *pandas* (versão 2.2.1) e *numpy* (versão 1.26.4) serão adotadas para manipulação matricial e estruturação dos dados. O ambiente de hardware consistirá em uma estação de trabalho equipada com processador Intel Core i7 (11^a geração), 16 GB de memória RAM e sistema operacional Windows 11 de 64 bits [KINA et al., 2026]. Por fim, em conformidade com as diretrizes de transparência científica, o código-fonte completo estruturado, o arquivo de dependências (*requirements.txt*) e as grades parametrizadas do *grid search* serão disponibilizados publicamente em um repositório da plataforma GitHub, permitindo a auditoria e replicação integral do pipeline metodológico estabelecido nesta pesquisa.

Referências Bibliográficas

- Ahmed, I.; Brahmacharimayum, A.; Ali, R. H.; Khan, T. A. & Ahmad, M. O. (2025). Explainable ai for depression detection and severity classification from activity data: Development and evaluation study of an interpretable framework. *JMIR Mental Health*, 12(1):e72038.
- Alghazzawi, D.; Ullah, H.; Tabassum, N.; Badri, S. K. & Asghar, M. Z. (2025). Explainable ai-based suicidal and non-suicidal ideations detection from social media text with enhanced ensemble technique. *Scientific Reports*, 15(1):1111.
- Allam, H.; Davison, C.; Kalota, F.; Lazaros, E. & Hua, D. (2025). Ai-driven mental health surveillance: Identifying suicidal ideation through machine learning techniques. *Big Data and Cognitive Computing*, 9(1):16.
- Alsaeedi, A. & Khan, M. Z. (2019). A study on sentiment analysis techniques of twitter data. *International Journal of Advanced Computer Science and Applications*, 10(2).
- Angiani, G.; Ferrari, L.; Fontanini, T.; Fornacciari, P.; Iotti, E.; Magliani, F.; Manicardi, S. et al. (2016). A comparison between preprocessing techniques for sentiment analysis in twitter. *KDWeb*, 1748:1–11.
- Azevedo, J. P. C.; de Oliveira, A. C. & Teles, A. S. (2024). Identificação de ideação suicida em textos usando aprendizado semi-supervisionado. *Journal of Health Informatics*, 16(Especial).
- Bashir, D.; Kaur, N. & Kumar, P. (2024). Suicide ideation detection using ml and dl algorithms assisted by nlp techniques. In *2024 5th International Conference on Recent Trends in Computer Science and Technology (ICRTCST)*, pp. 350–355. IEEE.
- Beriña, J. (2025). A natural language processing model for early detection of suicidal ideation in textual data. *Journal of Information Systems Engineering and Management*, 10:422–432.
- Bhuiyan, M. I.; Kamarudin, N. S. & Ismail, N. H. (2025). Detecting suicidal ideation in text with interpretable deep learning: A cnn-bigru with attention mechanism.

- In *2025 IEEE 9th International Conference on Software Engineering & Computer Systems (ICSECS)*, pp. 181–186. IEEE.
- Blanca Casas, S. M. & Guibert Reyes, W. (1998). Identificación de indicadores suicidas en pacientes generales por la enfermera de atención primaria. *Revista Cubana de Enfermería*, 14(2):117–123.
- Brown, L. A.; Boudreaux, E. D.; Arias, S. A.; Miller, I. W.; May, A. M.; Camargo Jr, C. A.; Bryan, C. J. & Armeý, M. F. (2020). C-ssrs performance in emergency department patients at high risk for suicide. *Suicide and Life-Threatening Behavior*, 50(6):1097–1104.
- Chen, T. & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pp. 785–794.
- Chowdhury, F.; Scoppola, C.; Parlatini, V.; Cortese, S. & Bellato, A. (2026). Association between suicidal thoughts and behaviours and markers of autonomic functioning and regulation in adults: A systematic review and meta-analysis. *Neuroscience & Biobehavioral Reviews*, p. 106672.
- Cortes, C. & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20:273–297.
- de Oliveira, A. C.; Diniz, E. J.; Teixeira, S. & Teles, A. S. (2022). How can machine learning identify suicidal ideation from user’s texts? towards the explanation of the boamente system. *Procedia Computer Science*, 206:141–150.
- Devlin, J.; Chang, M.-W.; Lee, K. & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186.
- Diniz, E. J.; Fontenele, J. E.; de Oliveira, A. C.; Bastos, V. H.; Teixeira, S.; Rabêlo, R. L.; Calçada, D. B.; Dos Santos, R. M.; de Oliveira, A. K. & Teles, A. S. (2022). Boamente: a natural language processing-based digital phenotyping tool for smart monitoring of suicidal ideation. In *Healthcare*, volume 10, p. 698. MDPI.
- Elsayed, N.; ElSayed, Z. & Ozer, M. (2024). Cautionsuicide: A deep learning based approach for detecting suicidal ideation in real time chatbot conversation. In *2024*

- IEEE 3rd International Conference on Computing and Machine Intelligence (ICMI)*, pp. 1–5. Ieee.
- Gil Arias, B. S.; Blandón Andrade, J. C.; Sidorov, G. & Morales-Ríos, A. (2026). Computational methods for the identification of suicidal ideation: a systematic review. *Frontiers in Artificial Intelligence*, 9:1704818.
- Hameed, S.; Nauman, M.; Akhtar, N.; Fayyaz, M. A. & Nawaz, R. (2025). Explainable ai-driven depression detection from social media using natural language processing and black box machine learning models. *Frontiers in Artificial Intelligence*, 8:1627078.
- Harris, Z. S. (1954). Distributional structure. *Word*, 10(2-3):146–162.
- Jianqiang, Z. & Xiaolin, G. (2017). Comparison research on text pre-processing methods on twitter sentiment analysis. *IEEE Access*, 5:2870–2879.
- Joinson, D.; Davis, O. & Simpson, E. (2024). The dynamics of emotion expression on twitter and mental health in a uk longitudinal study. *International Journal of Population Data Science*, 9(4).
- Jordan, D.; Cesar, V. & Carmo, L. d. (2022). Análise de técnicas de pln e machine learning na detecção de indícios de depressão.
- Jurafsky, D. & Martin, J. H. (2000). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. Prentice Hall, 3 edição.
- KINA, E.; Choi, J.-G.; Ishaq, A.; Shafique, R.; Villar, M. G.; Alvarado, E. S.; Diez, I. d. l. T. & Ashraf, I. (2026). Suicide ideation detection using social media data and ensemble machine learning model. *International Journal of Computational Intelligence Systems*.
- LeCun, Y.; Bengio, Y. & Hinton, G. (2015). Deep learning. *nature*, 521(7553):436–444.
- Levy, O.; Goldberg, Y. & Dagan, I. (2015). Improving distributional similarity with lessons learned from word embeddings. *Transactions of the association for computational linguistics*, 3:211–225.
- Liaw, A.; Wiener, M. et al. (2002). Classification and regression by randomforest. *R news*, 2(3):18–22.

- Liu, J.; Shi, M. & Jiang, H. (2022). Detecting suicidal ideation in social media: an ensemble method based on feature fusion. *International journal of environmental research and public health*, 19(13):8197.
- Long, S.; Cabral, R.; Poon, J. & Han, S. C. (2022). A quantitative and qualitative analysis of suicide ideation detection using deep learning. *arXiv preprint arXiv:2206.08673*.
- Lowry, N. J.; Goger, P.; Hands Ruz, M.; Ye, F. & Cha, C. B. (2024). Suicide risk screening tools for pediatric patients: a systematic review of test accuracy. *Pediatrics*, 153(3):e2023064172.
- Malhotra, A. & Jindal, R. (2024). Xai transformer based approach for interpreting depressed and suicidal user behavior on online social networks. *Cognitive Systems Research*, 84:101186.
- Manning, C. & Schutze, H. (1999). *Foundations of statistical natural language processing*. MIT press.
- Matrella, G.; Parada, G.; Mordonini, M. & Cagnoni, S. (2009). A video-based fall detector sensor well suited for a data-fusion approach. *Assistive Technology from Adapted Equipment to Inclusive Environments, Assistive Technology Research Series*, 25:327–331.
- Mokheleli, T.; Bokaba, T. & Mbunge, E. (2025). Explainable artificial intelligence for workplace mental health prediction. In *Informatics*, volume 12, p. 130. MDPI.
- Nadeem, G. (2025). Language as a lifeline: Leveraging nlp to suicide detection through context-aware ai models. *Journal of Computers and Intelligent Systems*, 3(4):312–318.
- Narayanan, V.; Arora, I. & Bhatia, A. (2013). Fast and accurate sentiment classification using an enhanced naive bayes model. In *International Conference on Intelligent Data Engineering and Automated Learning*, pp. 194–201. Springer.
- Narynov, S.; Mukhtarkhanuly, D.; Kerimov, I. & Omarov, B. (2019). Comparative analysis of supervised and unsupervised learning algorithms for online user content suicidal ideation detection. *J. Theor. Appl. Inf. Technol*, 97(22):3304–3317.
- Nações Unidas (2025). ‘shift the narrative’ on suicide to prevent loss of 720,000 lives annually.

- Negi, D.; Abhinandan; Jasveer & Taneja, S. (2023). Sentiment analysis of twitter data using kalman filters and lstm for political opinion prediction. *International Journal of Engineering Technologies and Management Research*, 10(6):58–67.
- Palomino, M. A. & Aider, F. (2022). Evaluating the effectiveness of text pre-processing in sentiment analysis. *Applied Sciences*, 12(17):8765.
- Pennington, J.; Socher, R. & Manning, C. D. (2014). Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pp. 1532–1543.
- Qorich, M. & El Ouazzani, R. (2024). Advanced deep learning and large language models for suicide ideation detection on social media. *Progress in Artificial Intelligence*, 13:135–147.
- Rabani, S. T.; Khan, Q. R. & Khanday, A. (2021). A nove approach to predict the level of suicidal ideation on social networks using machine and ensemble learning. *ICTACT J. SOFT Comput*, 11(2):2288–2293.
- Roaten, K.; Horowitz, L. M.; Bridge, J. A.; Goans, C. R.; McKintosh, C.; Genzel, R.; Johnson, C. & North, C. S. (2021). Universal pediatric suicide risk screening in a health care system: 90,000 patient encounters. *Journal of the Academy of Consultation-Liaison Psychiatry*, 62(4):421–429.
- Saif, H.; Fernández, M.; He, Y. & Alani, H. (2014). On stopwords, filtering and data sparsity for sentiment analysis of twitter. In *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC'14)*, pp. 810–817.
- Salton, G. & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information processing & management*, 24(5):513–523.
- Sayfullina, L. et al. (2014). Reducing sparsity in sentiment analysis data using novel dimensionality reduction approaches. *Aalto University*.
- Shingekar, S. R.; Rodriguez, V. J.; Bloom, P. A.; Yoo, D. W. & Saha, K. (2025). Interpersonal theory of suicide as a lens to examine suicidal ideation in online spaces. *arXiv preprint arXiv:2504.13277*.
- Shin, K. E.; Spears, A. P.; Zhang, R. & Cha, C. B. (2025). Suicide-related disclosure patterns among culturally minoritized youth: Examining differences across race, ethnicity, gender identity, and sexual orientation. *Suicide and Life-Threatening Behavior*, 55(1):e13026.

- Singh, J.; Beeche, C.; Shi, Z.; Beale, O.; Rosin, B.; Leader, J. & Pu, J. (2023). Batch-balanced focal loss: a hybrid solution to class imbalance in deep learning. *Journal of Medical Imaging*, 10(5):051809–051809.
- Souza, F.; Nogueira, R. & Lotufo, R. (2020). Bertimbau: pretrained bert models for brazilian portuguese. In *Brazilian conference on intelligent systems*, pp. 403–417. Springer.
- Steinkamp, J. & Cook, T. S. (2021). Basic artificial intelligence techniques: natural language processing of radiology reports. *Radiologic Clinics*, 59(6):919–931.
- Trentarossi, B.; Romão, M. E.; Barelo, S.; Baptista, M. N.; Visonà, S. D. & Belli, G. (2025). Digital detection of suicidal ideation: A scoping review to inform prevention and psychological well-being. *Behavioral Sciences*, 15(12):1601.
- Tripathy, A. & Rath, S. K. (2017). Classification of sentiment of reviews using supervised machine learning techniques. *International Journal of Rough Sets and Data Analysis (IJRSDA)*, 4(1):56–74.
- Turecki, G.; Brent, D. A.; Gunnell, D.; O'Connor, R. C.; Oquendo, M. A.; Pirkis, J. & Stanley, B. H. (2019). Suicide and suicide risk. *Nature reviews Disease primers*, 5(1):74.
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł. & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- Wang, S. I. & Manning, C. D. (2012). Baselines and bigrams: Simple, good sentiment and topic classification. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 90–94.
- Wu, H. C.; Luk, R. W. P.; Wong, K. F. & Kwok, K. L. (2008). Interpreting tf-idf term weights as making relevance decisions. *ACM Transactions on Information Systems (TOIS)*, 26(3):1–37.
- Xia, H. (2023). Continuous-bag-of-words and skip-gram for word vector training and text classification. In *Journal of Physics: Conference Series*, volume 2634, p. 012052. IOP Publishing.
- Xu, X.; Yao, B.; Dong, Y.; Gabriel, S.; Yu, H.; Hendler, J. et al. (2024). Mental-llm: leveraging large language models for mental health prediction via online text data. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 8(1):1–32.

- Yang, Z.; Leonard, R.; Tran, H.; Driscoll, R. & Davis, C. (2025). Detection of suicidal risk on social media: A hybrid model. In *International Conference on Advances in Social Networks Analysis and Mining*, pp. 38–49. Springer.
- Zareei, M.; Languré, A. D. L.; Enciso, F. A. A.; Garcia-Ceja, E. & Biswal, R. R. (2025). A comprehensive review of automatic methods for suicidal ideation detection.
- Zevallos, R. J.; Schoene, A. M. & Ortega, J. E. (2025). The first multilingual model for the detection of suicide texts. In *Proceedings of the Second Workshop on Scaling Up Multilingual & Multi-Cultural Evaluation*, pp. 1–11.

